Standardizing Revenue Forecasting at Rakuten Advertising: Fine-Tuning Time Series Foundation Models Across Industry Verticals

By

Harleen Buttar

Kevin Fan

Aren Mizuno

Kai Schindler

Supervisors: Anil Chaturvedi, Roger Moore

A Capstone Project

Submitted to the University of Chicago in partial fulfillment
of the requirements for the degree of

Master of Science in Applied Data Science

Division of Physical Sciences

August 2026

Abstract

Rakuten Advertising currently builds a separate revenue forecasting model per client with open-ended feature adjustment, producing inaccurate forecasts. This was addressed by benchmarking five time series foundation models, both zero-shot and under seven parameter-efficient fine-tuning methods, against statistical and machine-learning baselines on NielsenIQ receipt-level e-commerce data across three verticals. Covariate-aware, zero-shot Chronos was strongest in every vertical, cutting Weighted Absolute Percentage Error by roughly 45% against the best baseline. Adaptation helped in only 17% of runs and degraded the strongest backbones. The project delivers a standardized modeling and evaluation pipeline and a client-facing dashboard improving forecast accuracy, consistency, and interpretability.

Keywords: time series forecasting, foundation models, parameter-efficient fine-tuning, LoRA, affiliate marketing, consumer behavior data, revenue forecasting, interpretability

Executive Summary

Rakuten Advertising builds a separate forecasting model per client with open-ended feature adjustment, producing inaccurate, trust-eroding forecasts. This project replaces that with standardized, industry-specific models (Luxury Brand Retail, Luxury Department Retail, Travel) built on time series foundation models (TSFMs) and parameter-efficient fine-tuning (PEFT). Usable Finance data was absent from the NIQ feed, so results cover three verticals.

The approach evaluated five TSFMs (TimesFM, Chronos, MOMENT, Moirai, TTM), zero-shot and under seven adaptation methods, against statistical and ML baselines (SARIMA, SARIMAX, ETS, VAR, GARCH, LightGBM, XGBoost), trained on NielsenIQ receipt-level data with engineered temporal features, compared on nine error metrics (WAPE and MdAPE primary). This drew on literature reporting that pretrained TSFMs forecast strongly zero-shot, rivaling task-specific models, and that PEFT can match or exceed full retraining. The former held; the latter did not at this data scale. Covariate-aware Chronos, zero-shot, was strongest in every vertical, reducing mean WAPE by ~45% against the strongest baseline and exceeding the 13 to 25% target, while adaptation improved its base model in only 17% of runs and degraded the strongest backbones. Standardizing on this pipeline is projected to save $1.84M annually.

Deliverables include a scalable modeling and evaluation pipeline and a client-facing dashboard. Four next steps are recommended: process the additional Travel data; extend to Finance once data exists; parallelize model execution; and harden the dashboard for internal deployment. The pipeline's configurable design also supports extension to other NIQ categories, target metrics, and newer models.

Table of Contents

List of Figures

List of Tables

Introduction

Rakuten Advertising operates a large affiliate marketing network that connects advertisers with publisher partners across sectors such as luxury retail, finance, and travel. A core practice is building customized revenue forecasting models for each client. This forecasting is not a contractual obligation but a strategic tool: accurate projections help maintain strong client relationships and thus protect revenue. Clients value accurate forecasts as they directly impact revenue planning, budgeting, campaign timing, and partner selection. In the modern digital economy, firms possess large-scale consumer data to better understand purchasing behavior and optimize marketing strategies. Despite this, organizations like Rakuten Advertising still face challenges in effectively leveraging this information for accurate forecasting. Traditional time series models can be effective but are often too generalized to capture the complex, nonlinear patterns across diverse consumer segments (Zhang, 2003; Makridakis et al., 2018). Time series foundation models such as TimesFM offer a more flexible alternative: pretrained on broad data, they can forecast varied trends with little task-specific tuning (Bommasani et al., 2021; Das et al., 2024). Recent work in transfer learning, particularly parameter-efficient fine-tuning, suggests these models can be further adapted to client-specific contexts by incorporating narrower domain data, potentially improving accuracy and generalization (Hu et al., 2022; Liang et al., 2024). Whether that adaptation improves forecasts at the scale of Rakuten's per-vertical data is the central question this project examines.

Problem Statement

Currently, Rakuten Advertising builds a new model for each client and lets clients adjust features freely. This lack of standardization often produces inaccurate models, and when forecasts are unreliable, clients may lose confidence in the projects and reconsider the

partnership, putting revenue at risk. Additionally, at present, this manual process runs roughly 400 forecasts per month, each taking about 6 hours of analyst labor at $75/hr resulting in a cost of $2.16M annually. This project addresses these issues by fine-tuning time series foundation models on Nielsen (NIQ) Consumer Marketing Data, a large-scale receipt level consumer behavior dataset, to improve forecasting performance across luxury brand retail (both brand and department stores), finance, and travel, while also enhancing interpretability through user-facing analytical tools. By building a standardized model for each of its four focus industries, Rakuten Advertising can present clients with accurate, consistent forecasts they cannot distort, drawing in more revenue and strengthening credibility. Automating the bulk of this feature-tuning work could cut per-forecast time by roughly 85%, leaving around an hour per forecast for review and release, and bringing annual forecasting labor costs down to around $324K, a potential savings of about $1.84M per year. Ultimately, as the leading affiliate advertising firm, Rakuten Advertising aims to stay ahead of competitors by adopting cutting-edge models.

Analysis Goals

This project pursued several key outcomes. The first was a comprehensive understanding of the data landscape across all four verticals, both to inform model selection and to support a rigorous, multi-metric evaluation of time series foundation models that identifies top performers. Performance was evaluated on nine metrics (MAE, RMSE, MAPE, MdAPE, SMAPE, WAPE, pooled/weighted error across candidate models, R-squared, and forecast bias), with Weighted Absolute Percentage Error (WAPE) and Median Absolute Percentage Error (MdAPE) adopted as primary tracking metrics: together they give an aggregate, scale-independent view of accuracy across verticals while remaining robust to outliers and near-zero-revenue periods. A central goal was optimizing industry-specific performance by applying time series foundation models to each

vertical and specializing them through parameter-efficient fine-tuning, targeting a 13 to 25% reduction in WAPE against the statistical baseline. That range was derived from two benchmark results reported by Das et al. (2024) and Faw et al. (2025): on short-horizon Monash datasets, TimesFM base improves on the next-best baseline by roughly 7%, and in-context fine-tuning adds a further 6.8% over the base model, which compose multiplicatively to approximately 13%; on long-horizon ETT datasets, the adapted model improves on competing baselines by at least 25% directly. MdAPE was tracked as a secondary check on typical per-forecast accuracy.

The remaining goals were an interactive, client-facing interface that communicates forecasted advertising value, and a fully documented, scalable, repeatable pipeline extensible across industries and clients.

Scope

This project was scoped to four industry verticals: Luxury Brand Retail, Luxury Department Retail, Travel, and Finance. Forecasting draws on a large-scale NIQ dataset of US customer purchase, interest, and web-trends activity, spanning January 1, 2022 through April 5, 2026 and filtered and segmented by industry and brand prior to modeling. Usable Finance-vertical data was not available in the delivered feed, so reported results cover the remaining three verticals. An additional Travel dataset was also received but arrived in a different format and could not be integrated within the project timeline, leaving Travel results based on the primary data alone. With access to unmasked data, forecasting is conducted at both the industry-vertical level and the individual brand level, using daily aggregated data. This dual-granularity approach supports the project's original goal of building standardized models per vertical, while also allowing brand-level analysis where data permits.

That qualification matters, because brand-level analysis introduces a further limitation: not every brand has a transaction history long enough to support a full one-year test split. Brands with shorter or sparser histories are therefore excluded from brand-level modeling, evaluated on a reduced test window, or reserved for vertical-level aggregation. Brand-level findings are consequently not uniformly representative across all brands within a vertical, and conclusions at that level should be interpreted with this coverage limitation in mind.

In addition, compute was also a scoping constraint: no dedicated cluster or GPUs were provided, so the project was limited to methods that run affordably on available hardware. This ruled out full fine-tuning of the foundation models and motivated the focus on PEFT, which adapts a model by updating only a small fraction of its parameters.

Furthermore, agent-based exploration, in which a model would autonomously select and invoke tools during the forecasting process, was included in the original scope but was set aside as the project progressed.

Background

Affiliate marketing is a performance-based advertising model that differs from traditional channels like TV, radio, or display ads in two ways. First, it is pay-for-performance: advertisers pay publishers only when a defined action occurs, typically a click, lead, or sale, rather than paying for airtime regardless of outcome. Second, it reaches audiences through content creators, influencers, and online sites, whose recommendations sit inside relevant content. This makes affiliate placements more targeted and contextually relevant than mass advertising, and it has made affiliate marketing one of the fastest-growing segments. Networks like Rakuten Advertising sit at the center, connecting advertisers with publishers and handling the work of

tracking performance, attributing revenue, and forecasting advertising value for both sides. Since payouts and budgets depend on projected performance, accurate forecasting is a core input to pricing, budget allocation, and the trust that sustains these relationships.

Accurate revenue forecasting is therefore central to Rakuten Advertising's ability to deliver value across its affiliate network, which spans luxury retail, finance, and travel. Forecasting here is complicated by the heterogeneity of advertiser behavior: large retail clients show strong seasonal patterns and high data volume, while smaller advertisers may have sparse or irregular time series. The current practice of building a separate model per client with open-ended feature adjustment introduces inconsistency, reduces reliability, and erodes client trust. This review motivates a standardized TSFM-based alternative, adapted to each vertical through parameter-efficient fine-tuning.

Literature Review

Introduction

For decades, practitioners have relied on local statistical methods such as ARIMA and exponential smoothing, which are well-understood but require individual model fitting for each series and fail to generalize across domains (Box & Jenkins, 1968; McKenzie, 1984). Deep learning methods, including DeepAR (Salinas et al., 2020) and N-BEATS (Oreshkin et al., 2020), introduced globally trained models capable of capturing complex temporal dependencies across many series simultaneously, but still require substantial task-specific data and do not transfer out of the box to new domains. The emergence of large pretrained language models (Radford et al., 2019) has inspired an analogous paradigm in time series: foundation models pretrained on large, diverse temporal datasets that can be adapted to downstream tasks with

minimal additional training (Liang et al., 2024). This paradigm is particularly well-suited to Rakuten Advertising's operational context, where forecasting must generalize across four distinct verticals (Luxury Brand Retail, Luxury Department Retail, Travel, and Finance) without sacrificing domain-specific accuracy.

Challenges in Time Series Forecasting for Affiliate Marketing

Several structural challenges motivate the shift away from per-client, per-model development toward a standardized foundation model for each vertical, rather than a single model spanning all four.

Domain shift is among the most pervasive issues: differences in scale, seasonality, and consumer behavior across verticals cause models trained on one client or sector to degrade when applied to another. In the affiliate marketing context, a model well-calibrated to luxury fashion purchase cycles may perform poorly when applied to travel booking patterns or financial product interest signals, even when the underlying data structure is superficially similar.

Catastrophic forgetting compounds this challenge. When a model is sequentially fine-tuned on new client data, it risks overwriting patterns learned from prior datasets, which is a significant liability in a multi-client system where the model must continuously absorb new information without losing prior knowledge (Kirkpatrick et al., 2017).

Multi-scale temporal dynamics present a further challenge. Consumer and webtrends data contain patterns that operate simultaneously across short-term fluctuations, weekly promotional cycles, and long-term seasonal trends. Capturing these layers simultaneously is difficult,

particularly when a single model must generalize across the behavioral rhythms of luxury retail, travel booking, and financial products.

Finally, data scarcity affects portions of the client base. As Ansari et al. (2024) observe, the quality and quantity of public time series data is substantially more limited than in NLP, making it difficult to train robust models from scratch for smaller advertisers. These challenges collectively motivate the need for scalable pretraining combined with efficient, targeted fine-tuning.

Token-Like Embeddings and Transformer Architectures

Most contemporary TSFMs are built on transformer architectures, which require input in the form of structured embedding sequences rather than raw numerical values. In contrast to natural language, where tokens correspond to discrete words, time series data is continuous and requires a purpose-built representation strategy.

The most widely adopted approach is patch-based embedding, in which a time series is segmented into fixed-length non-overlapping (or overlapping) windows, each of which is projected into the transformer's embedding space. This strategy is central to models such as PatchTST (Nie et al., 2023) and TimesFM (Das et al., 2024), reducing the number of input tokens by a factor of the patch length and improving computational efficiency while preserving local temporal structure.

An alternative representation, adopted by Chronos (Ansari et al., 2024), treats time series values as discrete tokens through quantization: it converts continuous values into integer bin indices that any standard language model architecture can process natively. This allows Chronos

to leverage existing language model infrastructure without architectural modifications, at the cost of some resolution in the value space.

This mechanical distinction — continuous patches versus discrete quantized tokens — is the basis for the fine-tuning and PEFT-compatibility argument developed in the sections that follow.

Foundation Models for Time Series

Contemporary TSFMs split along a basic architectural fork with direct consequences for fine-tuning: patch-based models segment a series into fixed-length windows and project them into embedding space, while quantization-based models discretize continuous values into token indices so that an existing language model architecture can process them unmodified. This distinction matters beyond representation. Patch-based models (TimesFM, TTM) can typically be adapted by freezing the transformer core and updating only input/output projection layers, a narrow surgical intervention well suited to lightweight per-vertical adaptation. Quantization-based models (Chronos) may instead require vocabulary-level adjustment to handle domain-specific value distributions, a heavier intervention that complicates the kind of targeted covariate injection — competitor CPC series, promotional calendars — that Oldenburg et al. (2024) and Lok et al. (2025) show drives the largest accuracy gains in advertising-adjacent forecasting. This is a first, structural reason to weight patch-based architectures more heavily for Rakuten's covariate-dependent use case, independent of raw benchmark accuracy.

A second axis organizes the candidates by what this project actually needs: native probabilistic output (client-facing prediction intervals), covariate support (promotional calendars,

platform traffic), and resource efficiency across many brands and verticals. Measured against these three criteria, the field splits unevenly.

TimeGPT-1 (Garza et al., 2023), trained from scratch on over 100 billion points spanning finance, healthcare, IoT, web traffic, electricity, and retail, natively supports exogenous variables at inference and uses conformal prediction for uncertainty quantification — on paper, a strong fit. In practice, it is offered only as a hosted API rather than an open, fine-tunable checkpoint, which forecloses the client- and vertical-specific fine-tuning this project requires. It is retained here as an important benchmark reference point and as evidence that encoder-decoder architectures can handle covariates natively, but it is excluded from the empirical benchmarking stage on those grounds.

MOMENT (Goswami et al., 2024) is architecturally attractive for a different reason: its multi-task pretraining on the 13-million-series Time Series Pile supports forecasting, anomaly detection, classification, and imputation from a single backbone, which could in principle serve Rakuten's broader consumer-behavior analysis needs beyond forecasting alone. Two features weigh against it, however. Its pretraining corpus, at roughly 1.1 billion observations, is an order of magnitude smaller than Chronos's 84 billion or TimesFM's 100 billion, and — as the Moirai 2.0 ablation below suggests — corpus scale rather than parameter count appears to be the binding constraint on TSFM performance. More directly disqualifying for this project's client-trust motivation, MOMENT currently supports only point forecasts, with no native probabilistic output. It remains a useful multi-task reference point but is a weaker candidate for Rakuten's core forecasting deliverable than the probabilistic alternatives below.

Chronos (Ansari et al., 2024) and Moirai 2.0 (C. Liu et al., 2026) both generate native probabilistic forecasts, directly addressing the prediction-interval requirement. Chronos's TSMixup and KernelSynth augmentation strategies are particularly relevant to Rakuten's smaller advertiser segments, where Ansari et al.'s (2024) point about data scarcity applies most acutely; a model whose training procedure explicitly synthesizes additional series is better positioned to generalize to brands with limited history. Its parameter efficiency is notable on its own terms — Chronos-T5 (Mini, 20M) outperforms Moirai-1.0-R (Large, 311M) despite a smaller training corpus — but this observation sits in tension with the corpus-scale argument above, and is worth flagging as a genuine open question rather than resolving it prematurely: parameter count and corpus size are clearly not the whole story, and the identity of the pretraining corpus likely matters as much as its size. Moirai 2.0's own ablation, showing no performance gain from scaling 11.4M to 305M parameters, reinforces the corpus-first reading, but Chronos's result shows that a well-designed small model on a well-designed corpus can still outperform a larger model trained on a differently constructed one. This project's five-model, four-vertical benchmark design is itself a way of testing that question empirically rather than adjudicating it from prior literature alone.

Tiny Time Mixers (TTM) (Ekambaram et al., 2024) is the strongest fit against the resource-efficiency criterion specifically. At 1 million parameters and fine-tunable on a single GPU or CPU, it is the only model among those considered with an explicit exogenous mixer block for known-future covariates, directly applicable to campaign planning calendars, and it is the only architecture here to abandon self-attention entirely in favor of an MLP-Mixer design — a useful natural experiment on whether attention is necessary at all for this task, or whether the channel-independent-backbone-plus-channel-mixing-decoder design does the real work. Its

weakness is the same one MOMENT shares: no native probabilistic output, which will need to be addressed downstream (e.g., via quantile regression heads) if TTM is retained through to deployment.

Taken together, no single model satisfies all three criteria simultaneously: TimeGPT-1 and TTM handle covariates well but fail on fine-tunability or probabilistic output respectively; Chronos and Moirai 2.0 handle uncertainty quantification well but differ sharply in resource cost; MOMENT's multi-task flexibility does not compensate for its smaller pretraining corpus and point-forecast limitation. This is precisely the condition Tan et al. (2021) describe in the dynamic-pricing forecasting literature — no model dominates across all situations — and it is the empirical justification for benchmarking Chronos, TimesFM, TTM, MOMENT, and Moirai 2.0 against each other per vertical, rather than selecting a single model on the basis of literature review alone.

Parameter-Efficient Fine-Tuning

Full fine-tuning of all model parameters for each client is impractical at Rakuten's scale: it requires storing a separate model per client, incurs high computational costs, and is prone to overfitting when individual client histories are limited. PEFT addresses these constraints by freezing most of the pretrained model and updating only a small subset of parameters, preserving shared temporal representations while learning lightweight, client- or industry-specific adaptations (Lester et al., 2021). The seven techniques surveyed here were designed against a largely implicit assumption: a canonical transformer with self-attention layers and a token embedding table. The five TSFMs benchmarked in this project violate that assumption in different ways — TimesFM, Chronos, MOMENT, and Moirai 2.0 retain self-attention but differ

in embedding strategy (patch-based versus quantized-token), while TTM abandons attention entirely for an MLP-Mixer design. No single PEFT technique is therefore architecture-neutral across all five, and the choice of method has to be argued per model rather than applied uniformly.

Attention-targeting Methods

LoRA (Hu et al., 2022), its quantized variant QLoRA (Dettmers et al., 2023), and its accuracy-refined variant DoRA (S.-Y. Liu et al., 2024) all operate by decomposing or adapting the weight matrices inside attention and projection layers. This makes them a natural fit for TimesFM, Chronos, MOMENT, and Moirai 2.0, all of which retain conventional attention blocks, but they have no direct mechanism to attach to in TTM's MLP-Mixer backbone, where there are no query/key/value projections to decompose. LoRA is the most validated of the three for time series specifically: Gupta et al. (2024), as cited in Z. Liu et al. (2026), show LoRA fine-tuning of Lag-Llama, Moirai, and Chronos improves out-of-domain accuracy while substantially reducing compute relative to full fine-tuning, and Chronos's own authors suggest LoRA as a practical path to lowering fine-tuning cost. In the Rakuten context, this supports using LoRA as the default adaptation mechanism for the four attention-based models, learning lightweight industry-specific adjustments — the distinct promotional response curves of luxury department retail versus travel booking seasonality — without maintaining fully separate weights per vertical. QLoRA's marginal contribution here is memory rather than accuracy: it matters more for keeping adaptation cost low when running four vertical-specific fine-tuning jobs in parallel than for any accuracy gain over LoRA. DoRA is the strongest of the three on accuracy alone, improving over LoRA by roughly 3.7–4.4 points on commonsense-reasoning benchmarks in its original setting and composing with quantization as QDoRA to beat QLoRA outright,

which makes it the more defensible default where brand-level histories are short and every point of accuracy-per-parameter matters — at the cost of being a less battle-tested choice for time series specifically than LoRA. Prefix-tuning (Li & Liang, 2021), which prepends trainable vectors to key/value pairs at every attention layer, belongs in this same family for the same structural reason: it presupposes attention layers to prepend to, and so, like LoRA, is unavailable for TTM without modification.

Embedding-targeting Methods

Prompt-tuning (Lester et al., 2021) and p-tuning (X. Liu et al., 2021) instead prepend trainable soft tokens at the input embedding layer, which makes them more architecture-general in principle: every model in the benchmark has some form of input projection, whether TimesFM's and TTM's continuous patch embeddings or Chronos's discrete quantized token table. In practice, though, the semantics differ by embedding type. For Chronos, a soft prompt is genuinely token-like, sitting in the same discrete space the model was pretrained on. For TimesFM, TTM, and MOMENT, a "soft token" prepended to a sequence of continuous patch embeddings is a looser analogy, since there is no discrete vocabulary for it to share space with — the method transfers architecturally but not with the same conceptual cleanliness. These methods can be read as injecting industry-specific signal into the input sequence (purchase-pattern signals for luxury retail versus interest-rate sensitivity for finance) without touching internal weights at all, which is attractive for TTM specifically, since it sidesteps the attention-layer problem above. p-tuning's added complexity — a small trainable prompt encoder rather than directly learned vectors — buys flexibility at the cost of yet another component to train and validate per vertical, which is a harder case to make at Rakuten's deployment cadence than for prompt-tuning's simpler design.

Architecture-agnostic Fallback

BitFit (Ben Zaken et al., 2022), which trains only bias terms, is the one technique in this set with no dependency on attention or embedding structure — bias terms exist in linear layers regardless of whether they sit inside an MLP-Mixer or a transformer block, which makes it the only method here that applies to TTM without adaptation. Its cost is expressiveness: restricting updates to bias terms alone may be insufficient when adapting across industries as structurally distinct as travel and luxury retail, and it is a reasonable expectation, not yet confirmed by the benchmark, that BitFit will underperform LoRA-family methods on the four attention-based models precisely because it forgoes the higher-capacity weight-level adaptation those models can support.

What the Empirical Results Already Suggest

Fine-tuning results reported for the individual models are consistent with this per-architecture reasoning rather than a single universal recipe. TimesFM, fine-tuned only on its input and output residual blocks using 10% of training data, outperforms fully trained competitive models by 12–18% in MAE (Das et al., 2024) — a targeted, attention-adjacent intervention rather than a full LoRA sweep. Fine-tuned Chronos-T5 (Small) takes the top position on its benchmark, consistent with LoRA-style or lightweight full fine-tuning suiting its quantized token space. TTM, updating only its 0.3M-parameter head on 5% of training data, improves MSE by 15% over GPT4TS and 10% over Time-LLM without any attention-layer mechanism at all, which is itself evidence that architecture-appropriate PEFT — not a uniform LoRA-everywhere policy — is what makes fine-tuning effective across a heterogeneous model set. This motivates the project's design of matching PEFT technique to model architecture per

vertical, rather than applying a single method across all five TSFMs, and treats the resulting accuracy differences as informative about architecture fit rather than as noise to average over.

Post-Training: Collaborative and Reinforcement Fine-Tuning

Beyond supervised fine-tuning, a growing body of post-training strategies extends the adaptation toolkit for TSFMs (Z. Liu et al., 2026).

Collaborative fine-tuning at the parameter level encompasses LoRA and adapter-based approaches, with channel-aware LoRA variants shown to capture inter-variable dependencies while preserving training efficiency. At the model level, multimodal fine-tuning aligns time series encoders with language model text encoders via contrastive learning, enabling zero-shot transfer when labeled data is scarce, which is potentially relevant for new client onboarding at Rakuten. Knowledge distillation at the hybrid level transfers representational capacity from large teacher models into compact student models suited for deployment.

Reinforcement fine-tuning applies reward-based feedback rather than explicit supervision. The TimeHF pipeline (Qi et al., 2025) introduces time-series policy optimization (TPO), the first reinforcement-learning-from-human-feedback approach for time series, which incorporates expert preference feedback and reports a roughly 33% accuracy improvement over prior methods in a large-scale industrial deployment spanning long-tail and intermittent products. This is a finding with direct relevance to Rakuten's smaller advertisers where preference-aligned optimization may outperform standard supervised approaches.

Forecasting in Digital Advertising and Marketing Analytics

The preceding sections establish the technical foundations of TSFMs and PEFT. A smaller but more directly relevant literature applies time series forecasting inside digital advertising itself, and it converges on a practical lesson: in this domain, what a model is allowed to condition on matters as much as which model is chosen.

Cost forecasting in programmatic and performance-based channels has received the most attention. Oldenburg et al. (2024) benchmarked SARIMA, XGBoost, LSTM, and the Temporal Fusion Transformer (TFT) on daily cost-per-click using a Google paid-search dataset covering more than 249,000 German advertisers, testing each model on univariate features, on multivariate features, and on multivariate features augmented with competitors' CPC series grouped by time-series clustering. Competitor covariates mattered most at long horizons. At 60 days, the TFT with distance-clustered competitor series cut MAE from 0.307 to 0.232 (24%) and SMAPE from 0.241 to 0.196 (19%) relative to an otherwise identical univariate TFT, and improved on the SARIMA benchmark (MAE 0.306) by a comparable margin. The advantage widened under stress: across six travel advertisers in the COVID-19 recovery window, mean SMAPE was 0.329 with competitor covariates against 0.528 without, roughly 38% lower. Two implications carry over to this project. Competitive and cross-brand signals, not only a vertical's own history, may improve accuracy for the luxury retail and travel affiliates competing for attention within Rakuten's network; and the benefit concentrates exactly where forecasting is hardest, at long horizons and during demand shocks.

A parallel line of work addresses the publisher side. Würfel et al. (2021) applied the TFT to daily Google AdSense revenues for 42 publishers, combining each publisher's own traffic

characteristics with the mean revenue of its content category as a covariate. At a 30-day horizon the TFT recorded the lowest error among the models tested (MAE 14.77, SMAPE 0.356), against 15.82 for a Seq2Seq LSTM, 16.58 for DeepAR, and 23.99 for a plain LSTM, a 38% reduction over the weakest deep learning baseline. The margin over N-BEATS was negligible, however (MAE 14.84), and N-BEATS was more accurate at 14 days, so the case for the TFT rests on interpretability and covariate handling as much as on raw accuracy. Notably, the authors attribute the performance gain to the category mean revenue covariate, which their variable-importance weights rank among the most influential encoder inputs. The transferable claim is therefore narrower than a general endorsement of transformers: cross-entity covariates measurably improve forecasts even within a single advertising channel, which is the principle underlying Rakuten's cross-brand, cross-vertical design.

Model selection under heterogeneous data has been studied in adjacent settings. Tan et al. (2021) reviewed 84 research articles on time-series forecasting for dynamic pricing in digital signage advertising and proposed selecting a model from a series' linearity, stationarity, volatility, and length, concluding that no single model performs best across all situations. That conclusion frames the central design question this project tests empirically: whether one standardized model can serve every vertical, or whether vertical-specific and ultimately brand-specific selection is required.

Finally, advertising volume has long been understood in economics as strongly seasonal and cyclical. Dewenter and Heimeshoff (2017) modeled national advertising expenditure with a structural time series model that decomposes the series into trend, seasonal, and autoregressive components, establishing empirical precedent for the multi-scale temporal structure described above well before transformer-based foundation models existed.

Taken together, this literature suggests that forecasting within digital advertising and marketing analytics is not merely a generic time series problem transplanted into a new domain: it carries idiosyncratic covariate structures, including competitor behavior, publisher-specific seasonality, and campaign cyclicality, that general-purpose TSFM benchmarks do not typically evaluate. This gap is precisely what motivates the vertical- and brand-level benchmarking design adopted in this project.

Gaps and Implications for This Project

Several limitations in the current TSFM landscape bear directly on the design of this project. Covariate support and probabilistic-output support, discussed above, remain the two binding constraints on model selection: no candidate model satisfies both criteria alongside resource efficiency, which is the direct motivation for this project's per-vertical benchmarking design rather than a single-model recommendation.

Two further limitations, not yet addressed above, also bear on this project's design. Evaluation standardization is an active concern: Z. Liu et al. (2026) identify systematic inconsistencies in normalization, baseline selection, and metric computation across TSFM benchmarks. This project adopts consistent evaluation protocols, including fixed normalization, identical train/test splits, and a common set of accuracy metrics, across all four verticals to ensure fair cross-model and cross-industry comparison.

Data representativeness is rarely treated as a modeling constraint. TSFM benchmarks report accuracy without characterizing whose behavior the underlying series capture, and the same gap applies downstream: consumer panels enroll households by opt-in, and merchant coverage depends on network participation. Accuracy gains here should therefore be read as

improvements in predicting the observed panel rather than category-wide demand, which bounds how far these results generalize beyond Rakuten's network.

Interpretability remains underexplored across the models considered. TTM's channel-mixing attention weights have been shown to correlate with known domain relationships, and PCA projections of its backbone embeddings form distinct cyclic orbits reflecting underlying seasonality. Developing interpretability tooling on top of fine-tuned models, as planned for the client-facing dashboard in this project, aligns with an identified gap in the field and represents a meaningful contribution beyond benchmark performance.

Data

NielsenIQ (NIQ) is a leading consumer intelligence company, providing retail measurement and consumer panel data across global markets. The data used here is sourced from NIQ's US e-commerce receipt panel, delivered through Rakuten under its licensed institutional agreement, which enables analysis of merchant level revenue trends across verticals.

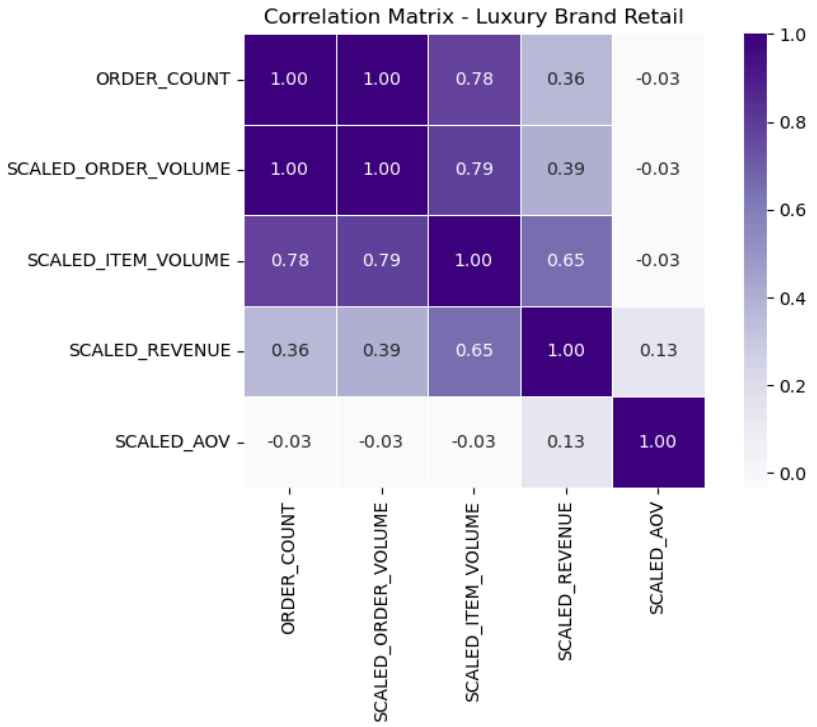
Data Sources

NIQ aggregates US e-commerce transaction data from two receipt-capture sources: Unroll.me, an inbox-management tool that captures purchase-confirmation emails from opted-in users, and Slice, a package-tracking app that extracts receipt data from user inboxes. These samples are projected to represent US e-commerce purchasing behavior as a whole, yielding metrics such as order volume, item volume, revenue, and average order value. The dataset covers US activity only and is not applicable to other countries. It spans January 2022 to April 2026 and includes major verticals such as Apparel & Footwear, Beauty, Department Stores, Electronics & Appliances, Entertainment, Food & Grocery, General Merchandise, Health & Wellness, Home &

Living, Jewelry & Accessories, Pet, Sporting Goods & Outdoor, and Travel. It contains both item-level and order-total revenue (order total includes sales tax and shipping), alongside market-projected (scaled) metrics representing estimated total US e-commerce activity.

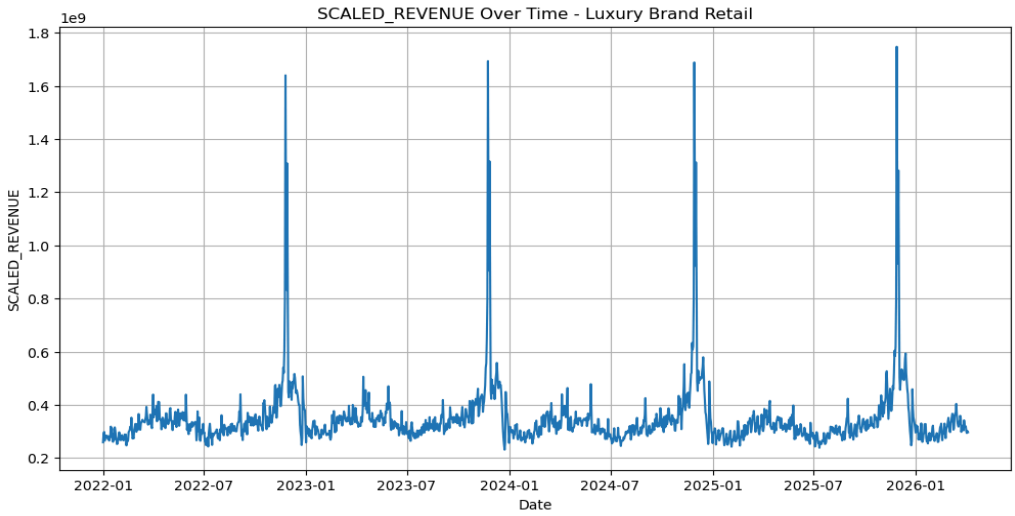
The dataset comprises 1,661,290 rows across 8 columns (~100MB), covering order date, vertical, merchant name, order count, and scaled order volume, item volume, revenue, and average order value (AOV). Exploratory analysis found no duplicate records, though scaled revenue and scaled AOV carried about 10% missingness, showing no strong association with observed variables and not concentrated within the primary verticals of interest. Of the verticals represented, the analysis narrowed to four: Luxury Brand Retail, Luxury Department Retail, Travel, and Finance. Finance was subsequently dropped, as no usable data for that vertical was present in the delivered feed. Revenue, order count, and order volume were right-skewed with many outliers across verticals. Order count, order volume, and item volume were strongly correlated with revenue, partly by construction since revenue scales with units sold, while AOV behaved more independently across verticals. Revenue and transaction metrics also showed clear seasonal trends, with holiday peaks each December and late November.

Figure 1: *Correlation Matrix Heatmap for Luxury Brand Retail*



Revenue and transaction metrics also showed clear seasonal trends, with holiday peaks each December and late November.

Figure 2: *Scaled Revenue Over Time*



Exclusions from this dataset stem from data-quality and licensing issues: NIQ may lack a contract to capture line-item data for a given merchant, or the merchant's data may break the aggregation process. These exclusions are applied at the source before the dataset is produced. Notable exclusions include Amazon and select grocery, beauty, and other retail merchants, which may cause undercounting in those verticals. For Walmart, item-level data is partial while order-total data is complete. Travel sub-type detail (Airlines, Lodging, OTAs, and similar) is not included in this pull, as Travel is captured at merchant level only. The final exclusion is non-US purchases.

Table 1. *Output Field Definitions*

Field	Description
order_date	Time period start date, truncated to the grain (month, week, or day)
vertical	Merchant category grouping assigned based on merchant_group and tags (e.g. Beauty, Department Stores)
merchant_name	Name of the individual retailer (or number for the encoded version)
order_count	Raw count of distinct orders observed in the NIQ panel – not projected
scaled_order_volume	Estimated total US market order volume, projected using each user's population weight
scaled_item_volume	Estimated total US market item volume, projected using population weights
scaled_revenue	Estimated total US market revenue at the item level (excludes tax/shipping), projected using population weights
scaled_aov	Average order value – calculated as scaled_revenue divided by scaled_order_volume

Descriptive Analysis

The raw panel reflects only the demographics of NIQ's capture sources, not the broader US shopping population, so NIQ assigns each user a population weight (projected_weight) calibrated to make the panel representative of US e-commerce shoppers across age, gender, income, and geography. In plain terms, the people whose receipts happen to be captured are not a perfect cross-section of everyone shopping online in the US, so each person's data is given a weight that says how many real-world shoppers they stand in for, much as a small taste-test panel is weighted so its results reflect an entire country's preferences. Since a single user appears across many order rows, user-level weights are deduplicated to one row per user before they are summed. When two distinct users happen to share the same projected weight, this deduplication can collapse them into a single record and understate the projection. To prevent that, a negligible tiebreaker derived from dense rank is appended to each weight, making every user's and every order's weight a distinct value so the deduplication treats them as separate units.

Figure 3. *User Unique Weight*

user unique weight = projected weight + (rank position × 0.0000001)

Figure 4. *Order Unique Weight*

unique order weight = projected weight + (rank position × 0.000000001)

Four scaled metrics are derived from these weights. Scaled order volume is the sum of order-level unique weights, giving the population-projected estimate of total transaction volume.

Scaled item volume sums the projected items field across transactions, and scaled revenue aggregates projected revenue. Scaled average order value (AOV) is the ratio of scaled revenue to scaled order volume, a population-weighted average rather than a simple mean over observed transactions. In plain terms, these four numbers answer how many orders happened, how many items were bought, how much was spent, and what the average order was worth, each projected to the full US shopping population rather than the smaller group in NIQ's raw data.

Figure 5. *Scaled Metrics*

scaled order volume = sum of unique order weight (across all orders)

scaled item volume = sum of projected items

scaled revenue = sum of projected revenue

scaled average order value = scaled revenue ÷ scaled order volume

Raw order count (order_count) is retained as a secondary diagnostic. Merchants or verticals with very low raw counts yield high-variance scaled projections and warrant caution regardless of the scaled figures. In plain terms, when only a handful of real orders underlie a projection, scaling them up can produce wildly unstable estimates, so a large scaled number built on thin data deserves extra skepticism even though the arithmetic still returns a value.

Scaled metrics are therefore the primary basis for inference throughout, since they represent projected market-level behavior and are the appropriate basis for trend analysis and externally reported estimates; raw counts are reported alongside as a check on sample adequacy.

Methodology

Feature Engineering

Raw order data was transformed into a model-ready dataset through a structured pipeline: transactions were segmented into target verticals, cleaned, aggregated to a daily grain indexed by order date, split chronologically into train and test partitions, and scaled. Features were then constructed on the resulting daily series across five categories: rolling statistics, lag features, rate-of-change features, ratio features, and temporal encodings.

Vertical Segmentation

Raw order records were rolled up into three analysis cohorts, defined by a fixed mapping from group key to display label and constituent vertical set: luxury brand retail (apparel & footwear, beauty, jewelry & accessories), luxury department retail (department stores), and travel.

Data Cleaning

Order date was parsed to datetime and used to derive year, month, and day of week. Exploratory analysis found no duplicate records, so no deduplication step was required. Scaled revenue and scaled aov carried roughly 10% missingness, with no strong association to other observed variables and no concentration within the primary verticals of interest, consistent with missingness that is effectively random rather than driven by a segment-specific pattern. On that basis, rows with a null in any column were dropped in full.

Aggregation and Calendar Completion

Cleaned order-level rows were aggregated to a daily grain by summing order count, order volume, item volume, and revenue by date. The result was reindexed to a continuous daily frequency, so calendar days with no recorded orders appear explicitly as zero-filled rows rather than being absent from the series. AOV was then recomputed from the cleaned daily aggregates as revenue divided by order count, with an explicit zero-order guard returning 0.0 rather than inf or NaN.

Train/Test Split and Scaling

The daily series was split chronologically: training covers January 1, 2022 through December 31, 2024 (1,096 observations), and the held-out test set runs from January 1, 2025 through April 5, 2026, with no shuffling, so no future information can leak into training through the split itself. A StandardScaler (default) or MinMaxScaler was fit on the training partition only and applied unchanged to the test partition; the date, year, month, and day-of-week columns were excluded from scaling.

Beyond the primary evaluation over the full test window, each saved forecast was additionally scored over its first 30, 90, 180, and 365 days to assess performance at shorter planning horizons without refitting: January 2025 alone, Q1 2025, H1 2025, and calendar year 2025, respectively. This horizon scoring covers only the first twelve months of the roughly fifteen-month test window.

Rolling Statistics

To capture short- and medium-term trend behavior, rolling means were computed over 7-day and 28-day windows for order count, order volume, item volume, revenue, and AOV.

Rolling standard deviations over the 7-day window were added as a proxy for recent volatility. All rolling computations used minimum periods of 1 so early observations are preserved rather than dropped as leading nulls.

Lag Features

Autoregressive structure was encoded via lag features at 1-, 7-, and 28-day offsets for revenue, order count, and AOV. The 1-day lag captures immediate carry-over effects, the 7-day lag captures same-weekday seasonality, and the 28-day lag approximates a monthly cycle.

Rate-of-Change Features

Day-over-day and week-over-week percentage changes were computed for revenue and order count, capturing momentum and short-term directional change that the level-based rolling and lag features alone do not represent.

Ratio Features

Two interaction ratios were derived: items per order (item volume divided by order count) and revenue per item (revenue divided by item volume). Zero denominators were replaced with nan prior to division, yielding nulls that are removed downstream rather than propagating spurious zeros or infinities.

Temporal Encodings

Day-of-week and month were encoded using sine and cosine projections, producing four cyclical features (dow_sin, dow_cos, month_sin, month_cos). Cyclical encoding is preferred over ordinal integers because it preserves metric continuity across period boundaries, so that, for

example, Sunday and Monday are treated as adjacent rather than maximally distant. A binary weekend flag was added to give the model a direct signal for the discrete behavioral shift observed on Saturdays and Sundays. Finally, an expanding mean of revenue was appended as a cumulative baseline reflecting the series' history up to each observation.

Modeling Frameworks

To build a standardized, scalable forecasting pipeline for affiliate marketing revenue, we evaluated three families of models: classical statistical models, machine learning models, and TSFMs. Each was tasked with forecasting daily affiliate marketing revenue (SCALED_REVENUE) across three business verticals: luxury retail, department retail, and travel. The objective was to produce accurate forward-looking estimates while improving scalability and reducing manual forecast adjustment. The families were chosen to isolate what actually drives forecasting performance on this data.

The statistical models (SARIMA, SARIMAX, Exponential Smoothing, VAR, and GARCH) test whether classical structure alone is sufficient, each targeting a different assumption about seasonality, exogenous signal, or volatility. The gradient-boosted trees (LightGBM and XGBoost) test the opposite extreme, whether a feature-rich learner with no pretraining and no stationarity assumptions can outperform that structure given the widest information set available. Together these seven form our baseline, each fit directly on the training data with no pretraining or transfer involved.

The five TSFMs (Chronos, TimesFM, Moirai, MOMENT, TTM) test whether pretraining on unrelated domains transfers here at all, and were selected to span distinct architectural and pretraining strategies rather than to sample one design repeatedly, ranging from decoder-only

autoregression to quantized tokenization to an attention-free mixer at a fraction of the parameter count. Selecting on architectural diversity means a negative result for one design does not generalize to the others, and a positive result can be attributed to something more specific than that foundation models work. Every TSFM result reported below, zero-shot or fine-tuned, is benchmarked against the baseline on identical held-out evaluation windows and error metrics. Fine-tuning was applied as frozen-base residual adapters using seven techniques in three groups: low-rank/reparameterization adapters (LoRA, QLoRA, DoRA), a selective bias-only adapter (BitFit), and prompt-based adapters (P-Tuning, Prompt Tuning, Prefix Tuning).

The models do not share a common information set. At the narrowest end, SARIMA, ETS, and MOMENT see only the target's own history. TimesFM, Chronos, and Moirai add eight deterministic calendar covariates (year, month, day-of-week, the cyclical sine and cosine encodings of the latter two, and a weekend flag), and each was additionally run in a no-covariate variant reducing to target history alone. SARIMAX and VAR condition instead on the business indicators (order count, order volume, item volume, and average order value), with SARIMAX treating them as exogenous regressors and VAR modeling them jointly as endogenous variables. TTM extends this to a broader exogenous set, combining those indicators with engineered rolling, lag, and momentum features and cyclical time encodings. At the widest end, the gradient-boosted trees consume the full engineered feature set alongside their own internally generated lag, rolling, and calendar features. Results should therefore be read as comparisons of model-plus-information-set rather than of algorithm alone.

Our working hypothesis was that TSFMs adapted through PEFT would outperform both the baselines by 13 to 25%, on the reasoning that large-scale pretraining yields transferable

temporal representations that lightweight, targeted fine-tuning can specialize to each vertical more efficiently and accurately than training a model from scratch.

All twelve non-adapted models, the seven baselines and the five TSFMs run zero-shot, share a single training and evaluation protocol. Splits are strictly chronological: training covers January 1, 2022 through December 31, 2024 (1,096 observations), and the held-out test window runs from January 1, 2025 through April 5, 2026, so no future information leaks into training. When tuning is enabled, a validation slice is carved from the end of the training data alone, sized to the larger of 30 days or 15% of the training window's length, and capped at the length of the test set, leaving an earlier-versus-later split within the training partition. Hyperparameter selection is therefore chronological rather than k-fold or randomized, either of which would leak forward-looking information into model selection. Within that slice each model is grid-searched on validation RMSE over the per-model grids given in the subsections below; every candidate combination, its validation MAE and RMSE, and the winning configurations are logged per vertical, so the selection is auditable. The winning configuration is then refit on the full training set and forecast once over the true held-out test window, and the validation slice is never reused in reported test metrics. This tune-validate-refit-forecast cycle runs independently per vertical (luxury brand, luxury department, travel), so orders, lags, and context lengths may differ by vertical rather than being shared globally.

Seasonal Autoregressive Integrated Moving Average with eXogenous Regressors (SARIMAX)

SARIMA served as our primary univariate statistical benchmark. It extends the autoregressive-integrated-moving-average formulation, modeling each observation as a function of its own lags and past forecast errors with differencing applied to remove non-stationarity, by

adding seasonal autoregressive, differencing, and moving-average terms to capture the recurring weekly and monthly patterns typical of retail and marketing series:

$$\text{SARIMA}(p,d,q)(P,D,Q)s$$

The model assumes the series can be rendered stationary through differencing and that residuals are approximately independent and homoscedastic. Stationarity was assessed through visual inspection and differencing. SARIMAX extends SARIMA with exogenous regressors, letting the forecast condition on external explanatory variables in addition to the series' own autoregressive and seasonal structure; it was evaluated on the same splits and metrics as SARIMA.

Variables: SARIMA uses SCALED_REVENUE history only. SARIMAX adds ORDER_COUNT, SCALED_ORDER_VOLUME, SCALED_ITEM_VOLUME, and SCALED_AOV as exogenous regressors.

Grid (16 configurations): order ∈ {(1,0,0), (1,1,0), (1,1,1), (2,1,1)} × seasonal_order ∈ {(0,0,0,0), (1,0,1,7), (1,1,1,7), (1,1,1,30)}.

Vector Autoregression (VAR)

VAR captures relationships among multiple correlated series simultaneously, predicting revenue from both its own lags and the lagged values of related indicators:

$$Y_t = c + A_1 Y_{t-1} + A_2 Y_{t-2} + \ldots + A_p Y_{t-p} + \varepsilon_t$$

VAR assumes stationarity and linear cross-variable relationships. A Vector Autoregressive Moving Average (VARMA) variant was also implemented but diverged numerically and was excluded from the reported results.

Variables: SCALED_REVENUE, ORDER_COUNT, SCALED_ORDER_VOLUME, SCALED_ITEM_VOLUME, and SCALED_AOV, all treated as endogenous.

Grid (8 configurations): maxlags ∈ {1, 2, 3, 7} × trend ∈ {c, n}.

Generalized Autoregressive Conditional Heteroskedasticity (GARCH)

GARCH was used to model time-varying variance rather than the conditional mean:

$$\sigma\square^2 = \alpha_0 + \Sigma\ \alpha_i\varepsilon^2\square_{-i} + \Sigma\ \beta\square\sigma^2\square_{-}\square$$

The motivation was volatility clustering in affiliate activity, particularly around seasonal promotions and high-traffic events, where variance itself is non-constant. The model assumes residual volatility evolves according to prior shocks and prior volatility levels. Log-return transformations were used where appropriate, and scaling adjustments were applied to stabilize optimization.

Variables: SCALED_REVENUE history only.

Grid (16 configurations): p ∈ {1, 2} × q ∈ {1, 2} × mean ∈ {Constant, Zero} × dist ∈ {normal, t}.

Exponential Smoothing (ETS)

ETS was included as a lightweight benchmark that forecasts revenue as a weighted combination of error, trend, and seasonal components, with exponentially decaying weights on older observations. It makes no stationarity assumption and requires minimal tuning, providing a fast reference point against the more heavily parameterized SARIMA-family, GARCH, and TSFM models.

Variables: SCALED_REVENUE history only.

Grid (16 configurations): trend ∈ {None, additive} × seasonal ∈ {None, additive} × seasonal_periods ∈ {7, 30} × damped_trend ∈ {False, True}.

Gradient-Boosted Trees (LightGBM, XGBoost)

LightGBM and XGBoost were trained on the full engineered feature set to provide a non-linear machine learning baseline distinct from both the classical statistical models and the TSFMs. Unlike SARIMA, VAR, and GARCH, they make no explicit stationarity or error-distribution assumption, instead learning non-linear feature interactions directly from the training data.

Variables: full engineered set (7- and 28-day rolling means, 7-day rolling standard deviation, lags at 1, 7, and 28 days, day-over-day and week-over-week changes, items-per-order and revenue-per-item ratios, cyclical day-of-week and month encodings, weekend flag, expanding revenue mean), plus internally generated lag, rolling, and calendar features.

Grid: n_estimators ∈ {300, 500} × learning_rate ∈ {0.03, 0.05} × tree-depth or leaf parameters.

TimesFM

A decoder-only transformer developed by Google Research, pretrained on roughly 100 billion real-world time points (primarily Google Trends and Wikipedia page-view series) supplemented with synthetic data (Das et al., 2024). It processes the series as contiguous, non-overlapping patches and generates forecasts autoregressively, using self-attention to capture long-range dependencies without relying on stationarity assumptions:

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d\_k})V$$

It produces point forecasts and, being natively univariate, cannot ingest the business-indicator regressors used by SARIMAX and VAR.

Variables: SCALED_REVENUE context window plus the eight deterministic calendar covariates. Also run in a no-covariate variant using the revenue context alone.

Grid (4 configurations): max_context ∈ {256, 512} × max_horizon ∈ {64, 128}.

Chronos

A framework from Amazon that reframes forecasting as language modeling (Ansari et al., 2024). It scales and quantizes real-valued observations into a fixed token vocabulary and trains T5-family (encoder-decoder) architectures on those tokens with a cross-entropy objective, deliberately ignoring explicit time or frequency features. Because it samples from the learned

token distribution, Chronos yields probabilistic rather than merely point forecasts, giving predictive intervals alongside the central estimate.

Variables: SCALED_REVENUE context window plus the eight deterministic calendar covariates. Also run in a no-covariate variant using the revenue context alone.

Grid (up to 9 configurations): context_length ∈ {256, 512, model default} × num_samples ∈ {50, 100, model default}.

Moirai

A masked-encoder universal forecasting transformer from Salesforce AI Research, pretrained on the Large-scale Open Time Series Archive (LOTSA), over 27 billion observations across nine domains (Woo et al., 2024). It is built for cross-frequency, any-variate forecasting: multiple patch-size projection layers handle different sampling frequencies, and an any-variate attention mechanism flattens multivariate inputs into a single sequence, so unlike TimesFM it can natively incorporate multiple related variates. It optimizes a mixture-distribution log-likelihood, making it probabilistic.

Variables: SCALED_REVENUE context window plus the eight deterministic calendar covariates, which Moirai ingests natively as additional variates. Also run in a no-covariate variant using the revenue context alone.

Grid (4 configurations): context_length ∈ {256, 512} × num_samples ∈ {50, 100}, with patch_size fixed to "auto".

MOMENT

A family of open, general-purpose foundation models from Carnegie Mellon (Goswami et al., 2024), pretrained on the "Time-series Pile" corpus using a masked-reconstruction objective on an encoder-only transformer. Rather than being forecasting-specific, MOMENT learns general representations supporting forecasting, classification, anomaly detection, and imputation through task-specific heads; it was used with a forecasting head in the project. Its reconstruction-based pretraining is a different inductive bias from the autoregressive generation of TimesFM and Chronos.

Variables: SCALED_REVENUE context window only. MOMENT was run univariate throughout, with no covariate variant.

Grid (2 configurations): context_length ∈ {256, 512}, with forecast_horizon fixed at 96.

Tiny Time Mixers (TTM)

A compact pretrained model from IBM Research, starting from roughly one million parameters (Ekambaram et al., 2024). Rather than self-attention, it is built on the lightweight TSMixer (MLP-Mixer) architecture, with innovations including adaptive patching, diverse-resolution sampling, and resolution prefix tuning, and it supports channel correlations and exogenous signals. Its small footprint makes inference fast enough to run on CPU, which matters if the pipeline must scale across many verticals cheaply.

Variables: the widest set of any TSFM. SCALED_REVENUE context window plus the business indicators (ORDER_COUNT, SCALED_AOV), engineered rolling, lag, and momentum features, items-per-order ratios, and cyclical time encodings, all supplied as exogenous channels.

Grid (4 configurations): prediction_length ∈ {96, 192, 336, 720}, with context_length fixed at 512.

Parameter-Efficient Fine-Tuning (PEFT)

PEFT methods freeze the pretrained backbone and train a small set of additional or selected parameters *inside* the network. The seven techniques selected were:

Table 2. *Summary of PEFT Methods and Their Underlying Mechanisms*

Technique	**Mechanism**
LoRA	Rank-*r* decomposition matrices injected as an additive update to frozen weight matrices, mergeable at inference
QLoRA	LoRA applied over a 4-bit NormalFloat-quantized frozen backbone, with double quantization and paged optimizers
DoRA	Frozen weights decomposed into magnitude and direction, with LoRA applied to the directional component
BitFit	Trains only the bias terms, freezing all weight matrices
Prefix-tuning	Trainable continuous vectors prepended to the key/value pairs at every attention layer
Prompt-tuning	Trainable soft tokens prepended at the input embedding layer only
P-tuning	Soft prompts generated by a small trainable prompt encoder rather than learned directly

None of these could be applied as perfectly specified. Each requires hooking trainable modules into designated internal layers of the backbone and backpropagating through it, and the five TSFMs expose neither a common layer interface nor, in several cases, the architecture the

method presupposes: prefix-tuning operates on self-attention, which TTM does not have, QLoRA requires control over the backbone's weight quantization, and the released inference APIs for several models return forecasts without exposing intermediate activations or gradients.

Therefore the models were adapted at the output level instead, implementing each of the seven as a residual correction head: the pretrained backbone is queried as a frozen black box for its zero-shot forecast, and a small trainable module of the corresponding form (a rank-8 low-rank projection for LoRA and its variants, a bias-only network for BitFit, soft-prompt fusion for the prompt-based methods) learns to correct that forecast toward the observed values. This preserves the practical goal of cheap, per-vertical specialization on top of a shared pretrained forecaster, and it preserves the parameter-count contrast between the techniques.

Training slides a context length of 128 and a horizon of 24 days over the training series (capped at 25 windows), generates the frozen backbone's zero-shot forecast for each window, and optimizes the head for 3 epochs (AdamW, MSE loss, batch size 8, learning rate 1e-3, dropout 0.05) against the true future values. These settings are constant across all seven heads and all three verticals.

Evaluation Procedure

Every model is scored on the same held-out test window under the chronological protocol described above: hyperparameters are selected on a validation slice drawn from the end of the training partition, the winning configuration is refit on the full training data, and a single forecast is produced over the test window, which is never touched during tuning. Training covers daily observations from January 1, 2022 through December 31, 2024 (1,096 observations); the held-out test window runs from January 1, 2025 through April 5, 2026. Beyond the primary

evaluation over the full window, each saved forecast is additionally scored over its first 30, 90, 180, and 365 days, corresponding to January 2025, Q1 2025, H1 2025, and calendar year 2025 respectively, so that accuracy at shorter planning horizons can be assessed without refitting. This horizon scoring covers the first twelve of the roughly fifteen months in the test window; the remaining months are included in the primary evaluation but not broken out by horizon.

Forecasts were scored on nine metrics spanning three families:

Scale-dependent: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) report error in the units of the target. Because SCALED_REVENUE is transformed, these are interpretable only as relative comparisons between models on the same vertical, not as absolute revenue amounts, and they are not comparable across verticals of differing scale. RMSE additionally penalizes large errors quadratically, making it the most sensitive of the set to individual bad days.

Scale-independent: Mean Absolute Percentage Error (MAPE), Median Absolute Percentage Error (MdAPE), Symmetric Mean Absolute Percentage Error (SMAPE), and Weighted Absolute Percentage Error (WAPE) normalize error against observed revenue, permitting comparison across verticals. They differ in how they aggregate: MAPE averages per-day percentage errors and is therefore dominated by days with small denominators; MdAPE takes the median and is robust to those outliers; SMAPE bounds the denominator using both actual and predicted values; and WAPE divides total absolute error by total actual revenue, weighting each day by its revenue contribution rather than treating all days equally.

Fit and calibration: R-squared reports variance explained relative to the test-window mean, and forecast bias reports mean signed error, distinguishing systematic over- from

under-forecasting. Bias is reported because two models can share an MAE while one is unbiased and the other consistently high, which matters for downstream revenue planning.

Cross-model aggregate: A pooled error is additionally computed across the full set of candidate models evaluated on a given vertical and window. Unlike the eight metrics above, this is not a property of any single forecast: it summarizes the candidate set as a whole, giving a reference level for how well the problem is solved by the available methods collectively rather than by the best of them. It is reported alongside the per-model table so that a leader's score can be read against the field rather than in isolation, and it provides a like-for-like basis for comparing difficulty across verticals, since a vertical where every model performs poorly is distinguishable from one where a single model happens to fail.

Of these, WAPE and MdAPE were selected as the primary tracking metrics. WAPE gives an aggregate, scale-independent measure that weights days by revenue contribution, so a model is judged mainly on the periods that carry the business, and it remains well defined when individual days are zero. MdAPE complements it by summarizing typical day-level accuracy without being distorted by a small number of extreme percentage errors.

Dashboard

To make the model comparisons described above reproducible and explorable beyond static notebook output, we developed an interactive web dashboard that runs the full forecasting pipeline live against user-specified configurations rather than serving precomputed results.

Architecture

The dashboard follows a two-tier architecture: a Python backend and a JavaScript frontend communicating over a REST API. The separation was a methodological choice rather than an implementation convenience. The comparison pipeline loads multi-hundred-megabyte pretrained weights for the five foundation models and performs live inference, and the residual correction heads must be trained on demand against whichever backbone a user selects. Neither step can be precomputed and served as a static asset. Both require a persistent compute environment able to execute the same Python modules used throughout the rest of this study (the data pipeline, feature engineering, model registry, and adapter training routines) without reimplementing that logic in a second language.

The backend is implemented in FastAPI as a thin orchestration layer over the existing pipeline. It reimplements no modeling logic, instead exposing the same forecast execution path used in standalone experimentation as a set of HTTP endpoints, so any result produced through the dashboard is methodologically identical to one produced by invoking the pipeline directly. The frontend is a single-page application built with React and Vite, using Recharts for visualization, and distributed as a Progressive Web App so it can be installed as a standalone application window independent of a browser tab.

Configuration and Scoping

The dashboard exposes four independent axes of configuration, each mapping onto an axis of the experimental design:

- **Vertical:** Luxury Brand Retail, Luxury Department Retail, or Travel, using the same groupings applied throughout the receipt-level analysis.

- **Brand:** An optional filter to a single merchant within the selected vertical. When a brand is selected, every chosen model runs twice, once against the vertical-wide aggregate and once against that brand's own series, with both sets of results presented side by side. This supports the question of whether aggregate-level forecast accuracy generalizes to individual brands, or whether performance is sensitive to the volume and volatility characteristics of a single merchant's data.
- **Forecast window:** A continuous parameter between 14 and 365 days rather than a small set of fixed horizons, so the sensitivity of each model's accuracy to horizon can be examined at arbitrary granularity. The upper bound is constrained dynamically by the length of history available for the selected vertical or brand, preserving a minimum training window.
- **Model and adapter selection:** Any subset of the twelve implemented models (seven classical and ML baselines, five foundation models) may be run simultaneously, along with any subset of the seven adapter techniques. When one or more techniques are selected, each foundation model runs both in its uncorrected zero-shot form and once per selected technique, so the effect of technique is assessed against a same-window, same-data baseline rather than against a separately reported figure.

Comparison View

When a request produces more than one result, whether from multiple models, multiple correction variants of one model, or both, the dashboard aggregates them into a single comparison view. The metrics defined in the Evaluation Procedure above are computed for every result and displayed in a sortable table.

To support interpretation across a large result set, each result is additionally assigned a rank on each metric and these ranks are averaged into an equal-weighted composite used to order the table. The composite is a navigational aid rather than a formal model-selection criterion: the metrics it averages are not independent, since MAE, RMSE, WAPE, MAPE, MdAPE, and SMAPE all measure absolute error magnitude and will tend to agree, so the composite weights that dimension far more heavily than it weights calibration or bias. Ordering by WAPE or MdAPE directly, per the primary metrics identified above, is the appropriate basis for any substantive claim.

Where a fine-tuned variant and its base are both present, the percentage change in RMSE between them is computed directly, isolating the marginal effect of the fine-tuning from other sources of variation between models. Wall-clock runtime is recorded per request and reported alongside accuracy, since characterizing the accuracy-compute tradeoff between foundation models and classical baselines is a central motivation of the study rather than an afterthought.

Results are also rendered visually: an overlaid forecast chart showing training history, held-out actuals, and each model's forecast; a zoomed view restricted to the forecast window; and a residual plot of actual minus predicted over time, which separates systematic over- or under-forecasting from unstructured error. Prediction intervals (p10 to p90) can be displayed for one model at a time to avoid visual clutter when many are compared, and any model's line can be hidden from a chart without removing it from the underlying comparison.

The complete selection state (vertical, brand, forecast window, selected models and fine-tuning techniques) is encoded into the page URL, so a configuration can be saved, shared,

and later restored exactly. The comparison table exports as CSV and any chart as a PNG for downstream reporting.

Deployment

The dashboard is deployed as two independently hosted components, reflecting the architectural separation above. The frontend is deployed to Vercel, which builds the Vite and React application and serves it as a static, globally distributed PWA, which suits a client-side single-page application with no server-side rendering requirement. The backend is deployed to a persistent host rather than a serverless function. Serverless execution is poorly suited to this workload on two counts: foundation model inference requires loading large pretrained weights into memory, which is prohibitively slow to repeat on every cold start, and individual requests involving multiple models or correction-head training can run for several minutes, exceeding typical serverless execution limits. The frontend receives the backend URL through a build-time environment variable.

Findings

The final evaluation covered 71 model configurations per vertical, for a total of 213 forecast runs across the Luxury Brand Retail, Luxury Department Retail, and Travel verticals: seven statistical and machine-learning baselines (SARIMA, SARIMAX, ETS, VAR, GARCH, LightGBM, and XGBoost), five zero-shot time series foundation models (TimesFM, Chronos, MOMENT, Moirai, and TTM) run in eight backbone configurations — the three models whose architectures accept exogenous covariates (Chronos, TimesFM, and Moirai) in both a covariate-aware and a no-covariate form, plus MOMENT and TTM, which accept none — and 56 fine-tuned variants produced by applying seven parameter-efficient fine-tuning methods

(LoRA, QLoRA, DoRA, BitFit, prompt tuning, prefix tuning, and P-tuning) to each of those eight backbones. A final round of experiments added exogenous covariates (order volume, item volume, and average order value) to the foundation models whose architectures accept them (Chronos, TimesFM, and Moirai); for those models, both a covariate-aware and a no-covariate configuration were evaluated, making covariate use an explicit comparison dimension rather than a fixed choice. Every configuration was trained and tested on the identical chronological split described in the Methodology, and all error metrics were computed from a single shared implementation so that no model benefited from inconsistent metric definitions. Results are reported primarily in WAPE and MdAPE.

Covariate-aware, zero-shot Chronos was the strongest forecaster overall. It achieved the lowest WAPE in all three verticals (40.6% in Luxury Brand Retail, 37.5% in Luxury Department Retail, and 80.3% in Travel), the lowest MdAPE in Luxury Brand and Luxury Department Retail (covariate-aware, zero-shot TimesFM edged it out in Travel, 81.5% against 83.3%), the best average rank across all eight evaluation metrics and all verticals (2.25, versus 4.79 for the next-best configuration), and the highest R-squared in every vertical (up to 0.73 in Luxury Department Retail). Relative to the strongest statistical baseline (ETS, mean WAPE 96.7%), covariate-aware Chronos reduced mean WAPE by roughly 45%, and by 52% relative to SARIMA, well beyond the project's 13-25% improvement target. No-covariate, zero-shot TimesFM was the second-strongest configuration (WAPE of 55.1%, 64.0%, and 86.5%). Table 2 summarizes the primary metrics for representative configurations, and Figure 4 shows the overall ranking.

Table 3. *WAPE and MdAPE (%) by model and vertical; best value per column in bold. "no cov" marks runs without exogenous covariates*

Model	WAPE			MdAPE		
	Lux. Brand	Lux. Dept	Travel	Lux. Brand	Lux. Dept	Travel
SARIMA	100.5	126.1	105.2	100.0	131.8	130.3
ETS	81.3	94.9	113.8	52.1	92.3	163.4
VAR	100.8	100.2	97.3	100.9	100.0	99.8
GARCH	320.4	87.6	665.4	274.0	77.1	1031.6
LightGBM	273.8	179.1	575.1	126.3	69.4	1090.4
XGBoost	167.8	220.0	686.0	236.1	68.9	1291.9
Chronos (covariates)	**40.6**	**37.5**	**80.3**	**28.6**	**31.1**	83.3
Chronos (no cov)	76.3	89.2	98.7	57.3	73.7	97.5
TimesFM (covariates)	99.8	117.6	85.6	94.3	118.5	**81.5**
TimesFM (no cov)	55.1	64.0	86.5	31.8	56.4	82.8
MOMENT	117.2	111.4	106.5	118.6	115.0	102.1
TTM	134.3	115.0	115.3	159.3	122.4	149.5
Moirai (covariates)	191.4	188.6	96.0	240.1	235.8	90.9
Moirai (no cov)	97.0	211.5	114.5	80.9	265.3	131.5

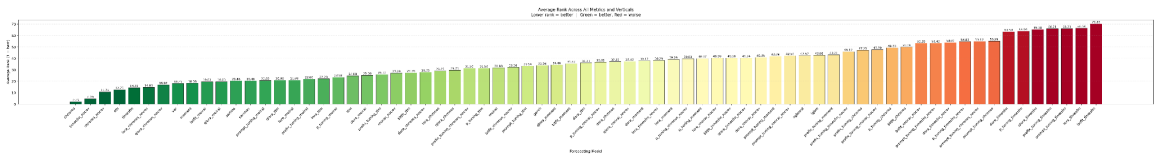
The covariate experiments produced the most consequential new finding of the project: whether exogenous covariates help is strongly model-dependent. The same three covariates cut Chronos's mean WAPE by 40% (from 88.1% to 52.8%) but made TimesFM 47% worse (from 68.5% to 101.0%) and Moirai 13% worse (from 141.0% to 158.7%), as summarized in Table 3. Covariates were therefore neither uniformly valuable nor uniformly safe: the information that transformed one architecture's forecasts actively disrupted another's. For deployment, this implies that covariate wiring must be validated per model family rather than adopted as a blanket

configuration, and the strongest overall system pairs each architecture with its best covariate setting rather than imposing one choice everywhere.

Table 4. *Effect of exogenous covariates on zero-shot foundation models (mean WAPE across verticals)*

Model	WAPE without covariates	WAPE with covariates	Effect
Chronos	88.1%	**52.8%**	improved 40%
TimesFM	**68.5%**	101.0%	degraded 47%
Moirai	141.0%	158.7%	degraded 13%

Figure 6. *Average rank across all eight evaluation metrics and all verticals for every model configuration (lower is better; green = better relative performance)*



The statistical baselines failed in ways that clarify what the foundation models were adding. VAR produced a WAPE close to 100% in every vertical (97.3% to 100.8%) with an R-squared near zero, which is what a forecast looks like when it collapses to the historical mean. SARIMA and SARIMAX returned identical results in all three verticals, indicating that the exogenous regressors contributed no signal beyond the series' own seasonal structure when attached to a classical model, in notable contrast to their large effect on Chronos. ETS was the strongest classical baseline (mean WAPE 96.7%), competitive in Luxury Brand Retail (81.3%) but fading at the Travel vertical and at long horizons. GARCH remained unstable across verticals (87.6% to 665.4%), and the gradient-boosted models again degraded badly out of sample (mean

WAPE of 343% for LightGBM and 358% for XGBoost), consistent with overfitting to training-period feature relationships.

The project's central hypothesis, that parameter-efficient fine-tuning would improve foundation-model accuracy beyond zero-shot performance, was again not supported, and the expanded evaluation strengthened the earlier conclusion. Across the 168 fine-tuned comparisons (seven methods applied to the eight backbone configurations across three verticals), only 17% improved WAPE relative to their own base model (21% of runs on the five default backbones, which carry covariates wherever the architecture supports them, against 10% on the three no-covariate ablations), and every method family had a negative median effect, from -51% (QLoRA) to -264% (prompt tuning), as shown in Table 4. The best-ranked fine-tuned configuration (LoRA on no-covariate Chronos) placed fifth overall, behind all four viable zero-shot configurations and the ETS baseline. The pattern observed in earlier rounds repeated: the largest gains occurred on the weakest TSFMs (fine-tunes of covariate-aware Moirai improved its mean WAPE by 23-24%), with the improved forecasts landing near the level of a mean-reverting forecast, consistent with the adapters acting as regularizers on erratic forecasts rather than learning vertical-specific structure, and with the data-scarcity interpretation: one aggregated daily series per vertical (roughly 1,100 training observations) is not enough for parameter-efficient fine-tuning to specialize a foundation model (Figure 5).

Table 5. *Effect of each fine-tuning method on WAPE relative to its own base model, across eight backbone configurations and three verticals (positive = improvement)*

Fine-tuning method	Median WAPE change	Runs improved
QLoRA	-50.8%	6 / 24
LoRA	-76.4%	5 / 24
P-tuning	-138.6%	3 / 24

DoRA	-165.2%	3 / 24
Prefix tuning	-189.9%	5 / 24
BitFit	-221.2%	4 / 24
Prompt tuning	-264.1%	2 / 24

Figure 7. *Improvement in Coefficient of Determination (R-squared) for each fine-tuned model relative to its own base model, averaged across verticals. Positive (green) bars indicate improvement; the y-axis is logarithmic beyond ±118% so bars of very different magnitude remain readable.*

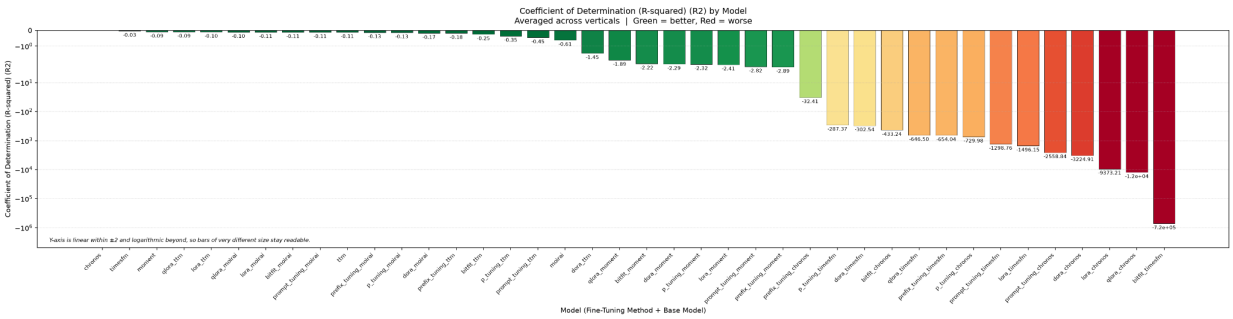


Figure 8. *Weighted Absolute Percentage Error (WAPE) by forecast horizon, averaged across the three verticals (lower is better). Covariate-aware Chronos is most accurate at every horizon (36.5%–55.1%), with no-covariate TimesFM second; LightGBM and XGBoost exceed the 260% axis cap at most horizons.*

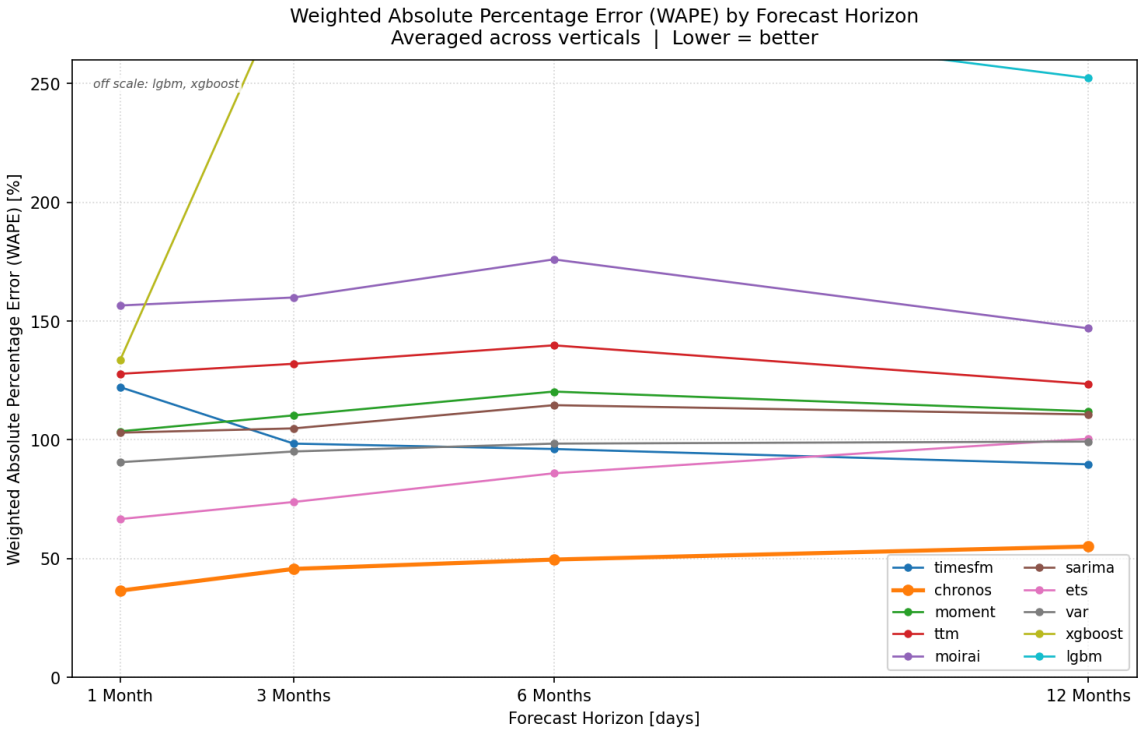
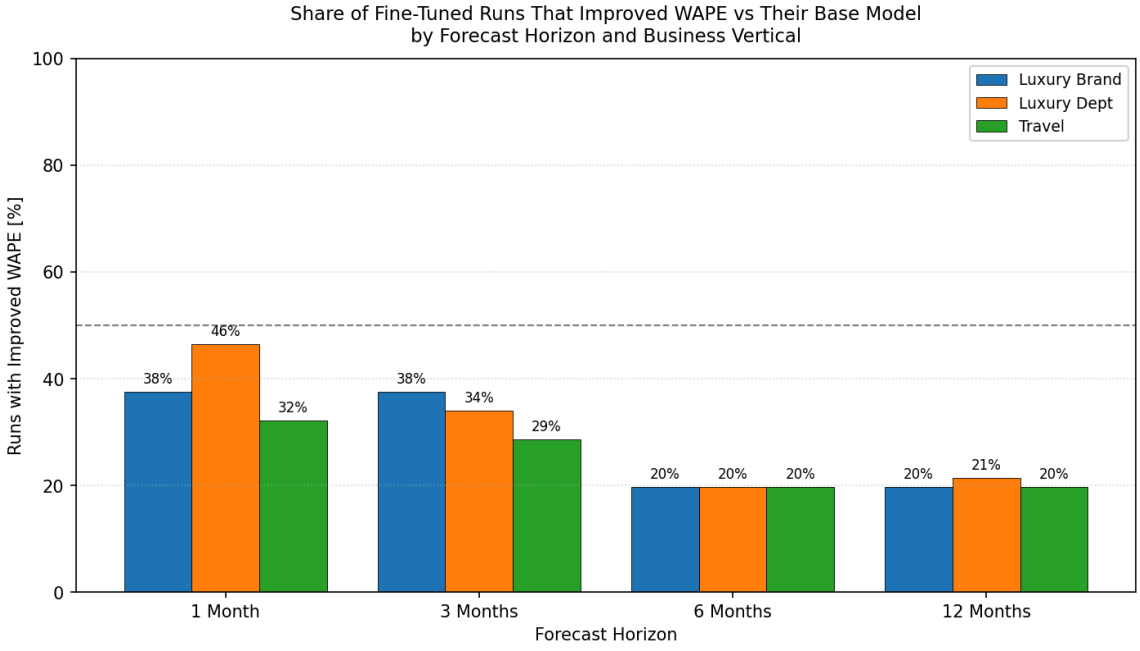


Figure 9. *Share of fine-tuned runs that improved WAPE over their own base model, by forecast horizon (168 comparisons each). Success falls from 39% at one month to 20% at twelve, never reaching the 50% break-even line (dashed).*



Finally, since Rakuten's forecasting requests span planning windows from monthly campaign adjustments to annual budgeting, we additionally evaluated every configuration at four forecast horizons (30, 90, 180, and 365 days) by scoring only the first N days of each saved forecast. These horizon figures therefore cover shorter windows than the full-test-window results in Table 2 and are not directly comparable to them. The ordering of the top configurations was stable across horizons: covariate-aware, zero-shot Chronos was the most accurate model at every horizon, from a mean WAPE of 36.5% at one month to 55.1% at twelve months, with no-covariate, zero-shot TimesFM second throughout (43.1% to 70.5%) (Figure 6). Error grows only moderately with horizon for the foundation models, while the classical baselines start near mean-forecast level even at one month. Fine-tuning was least harmful on short horizons (39% of fine-tuned runs improved WAPE at one month, falling to 20% at twelve months; Figure 7), consistent with the error-accumulation view of adapter damage, but at no horizon did fine-tuning help more often than it hurt. One caveat applies: the shorter windows cover single calendar periods (the one-month horizon is January 2025 alone), so short-horizon comparisons are indicative rather than seasonal averages.

Discussion

Objectives and Outcomes

This project met most of its stated objectives, and the gaps that remain follow either from deliberate decisions or from constraints outside the team's control. The work delivered the understanding of the data landscape called for in the Analysis Goals, implemented and compared twelve forecasting models under seven adaptation techniques using a consistent multi-metric

evaluation protocol, and produced a client-facing dashboard that operationalizes that comparison as a live, reusable workflow rather than a one-time notebook exercise.

The target of a 13 to 25% WAPE reduction relative to the statistical baseline was met and substantially exceeded: covariate-aware Chronos, used zero-shot, cut mean WAPE by roughly 45% against ETS, the strongest statistical baseline. Only half the hypothesis held, however. Foundation models did outperform the statistical baselines, but the advantage came entirely from those models used out of the box, before any adaptation was applied, rather than from the fine-tuning that was supposed to produce it.

Three elements of the original scope were not completed, for distinct reasons.

Agent-based exploration, in which a model would autonomously select and invoke tools during forecasting, was set aside as the project progressed. It was judged to add complexity without added value for the actual goal, since the dashboard already exposes every model and adaptation combination directly and interactively, giving analysts and clients a faster and more transparent route to the same understanding without introducing tool-selecting unpredictability into a client-facing revenue number. This was a considered scope reduction rather than a shortfall.

A further tranche of Travel-vertical data was received during the project but never processed. Integrating it would have required reconciling a format mismatch against the existing pipeline, and the remaining schedule did not allow for that work, so the Travel results rest on the original feed alone. This omission bears directly on the project's central negative finding. The most plausible explanation for the failure of adaptation is data scale, roughly 1,100 training observations per vertical, and the unprocessed travel data is the one available means of testing

that explanation. Whether a longer or denser series would change the adaptation result is therefore an open question rather than a settled one, and it is the first item that should be addressed in any continuation of this work.

The Finance vertical was dropped for a reason outside the project's control. Despite being named in the original scope as one of four target verticals, usable finance data was not available in the NIQ feed obtained for this project.

Strong TSFMs

Covariate-aware Chronos was the strongest configuration overall, achieving the lowest WAPE in all three verticals (40.6% in Luxury Brand Retail, 37.5% in Luxury Department Retail, and 80.3% in Travel) and the best average rank across all metrics and verticals. It also recorded the lowest MdAPE in Luxury Brand and Luxury Department Retail, though covariate-aware TimesFM edged it out in Travel (81.5% against 83.3%). No-covariate TimesFM was second overall (55.1%, 64.0%, and 86.5% WAPE, in the same vertical order). No adapted variant approached either.

The two leaders succeeded for different reasons, and the distinction matters for deployment. TimesFM's strength is best explained by domain similarity: its pretraining corpus is drawn primarily from Wikipedia page views and Google Trends, both of which share the shape of daily consumer revenue, namely continuous traffic-like metrics with strong weekly and holiday seasonality driven by aggregate human behavior rather than discrete events. That similarity plausibly accounts for why it transferred effectively on the univariate series alone. Chronos, by contrast, reached its result only once covariates were supplied, so its advantage rests on a broad pretraining corpus combined with access to contemporaneous business signal rather

than on domain match alone. Both results are evidence about these architectures on this specific task, not a general claim about how either ranks against other foundation models elsewhere.

These numbers need one piece of context. A WAPE of 100% means total absolute error equals total actual revenue, which is roughly what a forecast achieves by simply predicting the historical mean every day. Most configurations in this study scored at or above that level. Only two beat it by a clear margin: Chronos at a mean WAPE of 52.8% and no-covariate TimesFM at 68.5%. The 45% improvement over ETS is therefore real, but it was measured against a field in which almost nothing outperformed a mean forecast, and that context should travel with the headline numbers.

Negative Adapter Impact

The project's central hypothesis, that lightweight adaptation would improve on zero-shot performance, was not supported. Across 168 adapted comparisons, only 17% improved WAPE relative to their own backbone, every technique family had a negative median effect ranging from -51% (QLoRA) to -264% (prompt tuning), and the best adapted configuration ranked fifth overall, behind all four viable zero-shot configurations.

The most likely explanation is data scale rather than a flaw in the techniques. Each adaptation run had access to a single aggregated daily series per vertical, roughly 1,100 training observations, which is small even by the standards of methods designed for limited task-specific data. Where adaptation produced a change, it typically pulled the forecast toward the historical mean rather than learning vertical-specific temporal structure. This also explains where gains appeared: the largest improvements were on the weakest backbones, where a pull toward the mean is itself an improvement, while the damage was worst on TimesFM, whose zero-shot

forecast was already well calibrated and therefore had little room to improve and considerable room to be dragged toward a worse baseline. This explanation is also a limitation on what the result can support. Since the available data lacked the depth to test whether longer per-vertical series would behave differently, the negative adapter finding is bounded by data availability rather than established as a property of the methods, and it should not be extrapolated to settings with substantially more training data, for example brand-level series with several years of history, without direct testing.

Furthermore, the seven techniques evaluated here are not literal internal adaptations of each backbone's weights as they are implemented in the literature. Building genuine internal LoRA, BitFit, or prefix-tuning implementations across five architecturally dissimilar foundation models (TimesFM's decoder-only transformer, TTM's attention-free MLP-Mixer, Chronos's quantized token vocabulary, and Moirai's and MOMENT's encoder-based designs) was not feasible. Each technique was instead implemented as an external residual correction head that adjusts a frozen backbone's output. Such a head can only learn systematic bias in that output; it cannot alter the representations that produced it. That ceiling plausibly constrains how much adaptation could have helped, independent of data scale, but it equally constrains what the negative result demonstrates: these findings characterize this residual-correction formulation specifically, not parameter-efficient fine-tuning as the literature defines it. A project able to build model-specific internal adapters might reach different conclusions about which technique performs best, or about whether adaptation helps at all. The two constraints cannot be separated with the experiments run here.

Covariate Impact

The covariate experiments produced the most consequential finding of the project: whether exogenous covariates help is strongly model-dependent. The same three covariates cut Chronos's mean WAPE by 40%, from 88.1% to 52.8%, while making TimesFM 47% worse (68.5% to 101.0%) and Moirai 13% worse (141.0% to 158.7%). Covariates were therefore neither uniformly valuable nor uniformly safe: information that transformed one architecture's forecasts actively disrupted another's. The practical implication is that covariate wiring must be validated per model family rather than adopted as a blanket configuration, and the strongest overall system pairs each architecture with its own best covariate setting rather than imposing one choice everywhere.

Behavior Across Forecast Horizons

Every configuration was additionally scored at 30, 90, 180, and 365 days, reflecting planning windows that range from monthly campaign adjustment to annual budgeting. Three patterns emerged. First, the ranking of the top configurations was stable throughout: covariate-aware Chronos was most accurate at every horizon, from a mean WAPE of 36.5% at one month to 55.1% at twelve, with no-covariate TimesFM second in each case, so a single model can serve all planning windows rather than switching by horizon. Second, the two families degrade differently. Error grows only moderately for the foundation models, whereas the classical baselines begin near mean-forecast level even at one month and have little further to fall, so the gap between them widens with horizon and the case for a pretrained model is strongest where the business need is longest-dated. Third, adaptation was least harmful at short horizons, improving WAPE in 39% of runs at one month, 33% at three, and 20% at both six and

twelve, consistent with error accumulation as a correction learned on 24-day windows is extrapolated across a year, though at no horizon did adaptation help more often than it hurt.

One caveat bounds all three observations. The shorter windows cover single calendar periods, with the one-month horizon corresponding to January 2025 alone, so short-horizon results reflect the seasonal character of a specific month rather than an annual average.

Limitations

Three constraints bound the results, two of which were introduced above as explanations and are restated here for what they limit rather than what they account for:

- **Data coverage:** Results cover three of the four verticals named in the original Scope. Finance is absent entirely, so nothing here speaks to financial-product forecasting. Travel was evaluated on the original feed alone, without the additional and more granular data received later in the project, so the Travel results reflect a narrower view of that vertical than the data on hand would allow.
- **Adapter implementation:** Each adaptation run trained on a single aggregated daily series of roughly 1,100 observations. The finding that adaptation did not help may therefore reflect how little data was available rather than any weakness in the techniques. Furthermore, the seven techniques were built as external residual correction heads rather than as internal modifications to each backbone's weights. Internal implementation was not feasible because several of the five backbones expose inference APIs that return forecasts without access to intermediate activations or gradients, and some methods presuppose an architecture that a given backbone lacks, such as prefix-tuning's reliance on self-attention, which TTM's MLP-Mixer design does not have. The results therefore

describe the correction-head implementation. They are not direct evidence of parameter-efficient fine-tuning as the literature implements it, and a project able to build model-specific internal adapters might reach a different conclusion.

Dashboard hyperparameters: Hyperparameters for each model were selected during the study at fixed horizons, whereas the dashboard permits any horizon between 14 and 365 days. A forecast produced at a user-selected horizon therefore uses hyperparameters tuned at a different one, so dashboard results are appropriate for exploring relative model behavior across horizons but are not equivalent to a full retune at the selected horizon.

None of these limitations undermines the project's central practical finding, that pretrained foundation models used as delivered outperformed both the client's status quo and every adaptation attempted at this data scale. That finding should be read as a statement about this data, these three verticals, and this implementation, not as a general claim about time series foundation models or fine-tuning.

Conclusion

Summary of the Deliverable

The final deliverable provided to Rakuten Advertising is an interactive forecasting benchmarking dashboard, built on a reusable data and modeling pipeline, that allows a user to compare twelve forecasting approaches across the Luxury Brand Retail, Luxury Department Retail, and Travel verticals in Rakuten’s NIQ receipt-level dataset, at both the vertical-aggregate and individual-brand level. The twelve comprise seven statistical and machine-learning baselines

(SARIMA, SARIMAX, ETS, GARCH, VAR, LightGBM, XGBoost) and five time series foundation models (Chronos, TimesFM, TTM, MOMENT, Moirai), each of the latter also evaluated under seven residual head techniques (LoRA, QLoRA, DoRA, BitFit, P-Tuning, Prompt Tuning, Prefix Tuning).

Rather than requiring a stakeholder to run individual scripts or notebooks per model, the dashboard exposes the entire comparison as a live, point-and-click workflow. A user selects a vertical, optionally a specific brand, a forecast horizon between two weeks and one year, and any combination of models and correction techniques, then receives standardized accuracy metrics, runtime, and visual forecast and residual comparisons across every selected configuration in a single view. This serves two audiences. For technical stakeholders, it is a reusable experimentation surface for evaluating new models, verticals, or brands as they become relevant, without additional engineering work. The underlying pipeline (niq_eda_pipeline_flexible.py, feature_engineering.py, and the model registry) was built generically enough that adding a new vertical group, brand, or model implementation extends the dashboard automatically rather than requiring dashboard-specific changes. For non-technical stakeholders, such as planning or marketing teams deciding how much confidence to place in a given forecast, it provides a way to inspect model disagreement, forecast uncertainty, and historical residual bias for a specific brand or vertical directly, rather than relying on a single aggregate accuracy figure reported after the fact.

In both cases the dashboard is a decision-support tool rather than a source of immediate answers. It does not return a single recommended forecast or declare which model to trust. It assembles the evidence needed to make that judgment, showing where models agree, where they diverge, and how each has erred historically, and leaves the choice to the analyst. This reflects a

finding of the study itself: the best configuration is not constant across verticals or architectures, so a tool that returned one answer would obscure the variation a user needs to see.

Quantified Impact

The primary quantified impact is a projected reduction in forecasting labor. As established in the problem statement, the current process runs roughly 400 forecasts per month at approximately six hours of analyst effort each, at $75 per hour, or about $2.16M annually.

The bulk of that six hours is per-client feature adjustment: selecting and tuning inputs for each individual client series rather than producing the forecast itself. The finding that a single standardized zero-shot foundation model per vertical outperformed the client's per-client custom models, reducing mean WAPE by roughly 45%, removes the need for that adjustment step. If the standardized model is applied per vertical and analyst involvement is limited to reviewing and releasing the output, per-forecast time falls by approximately 85%, leaving around an hour per forecast for review, and lowering annual forecasting labor to roughly $324K for an estimated $1.84M in annual savings.

The specific accuracy improvements, and the absence of them where adaptation was applied, are reported in detail in the Findings section. In summary, covariate-aware Chronos used zero-shot achieved the lowest WAPE in all three verticals tested, exceeding the project’s 13 to 25% improvement target by a wide margin with a 45% reduction relative to ETS, while residual correction improved on its corresponding base model in only 17% of runs and degraded the strongest backbones most.

Independent of accuracy outcomes, the dashboard has a measurable process impact. Work that previously required manually invoking up to nineteen distinct model and correction

implementations across separate scripts, each with its own data loading, splitting, and metric computation, now reduces to a single request that runs an arbitrary subset of them as a parallel comparison, with standardized metrics and a shared caching layer that returns a previously computed configuration instantly rather than recomputing it. This collapses model evaluation from a multi-step, script-per-model process into a single repeatable and shareable workflow, since comparison configurations are encoded directly into a URL. That is the most durable contribution to Rakuten's ongoing workflow, independent of any one experiment's outcome.

Next Steps

Several extensions were identified during development but fell outside the scope of this capstone and are recommended as immediate next steps.

Processing Additional Travel Data

A further tranche of Travel-vertical data was received during the project but could not be integrated within the schedule because of a format mismatch against the existing pipeline. Reconciling that schema is the first item to address, both because it completes the Travel results and because it is the one available means of testing whether the negative adaptation finding reflects the techniques themselves or simply the limited training data available to them.

Extending to Finance Vertical

Extending to the finance vertical was scaffolded in the pipeline's vertical grouping but never populated with data during this study. This can be done once suitable NIQ data for that vertical becomes available.

Parallelizing Model Execution

Comparisons currently run sequentially on the backend, so a comparison involving several foundation models (not cached) can take several minutes. Running independent model evaluations concurrently, for example via a process pool, would reduce this substantially and is the most impactful near-term performance improvement.

Production Hardening and Automated Re-Evaluation

If the dashboard is adopted as a standing internal tool rather than a capstone artifact, two changes are required. First, it needs securing: authentication should be added, and Cross-Origin Resource Sharing, the browser mechanism that controls which web domains are permitted to call the backend, should be restricted to Rakuten's internal domain rather than left in the permissive any-origin configuration used during development. Second, comparisons should be re-run automatically as new NIQ data arrives, so that results reflect current data rather than requiring a manual re-run. Together these would evolve the dashboard from an on-demand comparison tool into a secured, lightweight, continuous model-monitoring surface.

Other Recommended Uses and Extensions

At a higher level, several extensions of this work may be of interest beyond the immediate next steps above:

Generalization to Other Product Categories

Since the pipeline was built around configurable vertical groupings and brand filters rather than hard-coded to the three verticals studied here, the same framework could be applied

to other NIQ-covered categories with minimal additional engineering, extending the benchmarking exercise beyond retail.

Extension to Additional Target Metrics

This study forecast scaled revenue; the same pipeline and dashboard could be applied with minimal modification to other available metrics such as order volume or average order value, to determine whether model rankings hold across business metrics or are specific to revenue forecasting.

Incorporation of Newer Foundational Models and Adaptation Methods

The model and technique registries were built as extensible mappings specifically so that new approaches can be added as they are released, without altering the dashboard or pipeline architecture. This makes the framework a natural fit for tracking a fast-moving research area over time rather than a fixed, one-time comparison.

References

Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S. S., Arango, S. P., Kapoor, S., Zschiegner, J., Maddix, D. C., Wang, H., Mahoney, M. W., Torkkola, K., Wilson, A. G., Bohlke-Schneider, M., & Wang, Y. (2024). Chronos: Learning the language of time series. *Transactions on Machine Learning Research*.

Ben Zaken, E., Goldberg, Y., & Ravfogel, S. (2022). BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. https://doi.org/10.18653/v1/2022.acl-short.1

Bommasani, R., et al. (2021). *On the Opportunities and Risks of Foundation Models.* arXiv:2108.07258. https://arxiv.org/abs/2108.07258

Box, G. E. P., & Jenkins, G. M. (1968). Some recent advances in forecasting and control. *Journal of the Royal Statistical Society. Series C (Applied Statistics),* 17(2), 91–109.

Das, A., Kong, W., Sen, R., & Zhou, Y. (2024). A decoder-only foundation model for time-series forecasting. *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 10148–10167.

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *arXiv*. https://doi.org/10.48550/arXiv.2305.14314

Dewenter, R., & Heimeshoff, U. (2017). Predicting advertising volumes using structural time series models: A case study. *Economics Bulletin, 37*(3), 1644–1652.

Ekambaram, V., Jati, A., Dayama, P., Mukherjee, S., Nguyen, N. H., Gifford, W. M., Reddy, C., & Kalagnanam, J. (2024). Tiny Time Mixers (TTMs): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*.

Faw, M., Sen, R., Zhou, Y., & Das, A. (2025). In-context fine-tuning for time-series foundation models. *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267, 16355–16374.[C1]

Garza, A., Challu, C., & Mergenthaler-Canseco, M. (2023). TimeGPT-1. *arXiv*. https://doi.org/10.48550/arXiv.2310.03589

Goswami, M., Szafer, K., Choudhry, A., Cai, Y., Li, S., & Dubrawski, A. (2024). MOMENT: A family of open time-series foundation models. *Proceedings of the 41st International Conference on Machine Learning, PMLR 235*.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences, 114*(13), 3521–3526. https://doi.org/10.1073/pnas.1611835114

Lester, B., Al-Rfou, R., & Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. *arXiv*. https://doi.org/10.48550/arXiv.2104.08691

Li, X. L., & Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. *arXiv*. https://doi.org/10.48550/arXiv.2101.00190

Liang, Y., Wen, H., Nie, Y., Jiang, Y., Jin, M., Song, D., Pan, S., & Wen, Q. (2024). Foundation models for time series analysis: A tutorial and survey. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 6555–6565.

Liu, C., Aksu, T., Liu, J., Liu, X., Yan, H., Pham, Q., Savarese, S., Sahoo, D., Xiong, C., & Li, J. (2026). Moirai 2.0: When less is more for time series forecasting. *arXiv*. https://doi.org/10.48550/arXiv.2511.11698[C1]

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-decomposed low-rank adaptation. *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 32100–32121.

Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., & Tang, J. (2021). P-tuning: GPT understands, too. *arXiv*. https://doi.org/10.48550/arXiv.2103.10385

Liu, Z., Li, B., Huang, H., Sun, Y., Wang, Y., Wu, M., & Ma, Q. (2026). From pre-training to post-training: A survey on time series foundation models. *TechRxiv*. https://doi.org/10.36227/techrxiv.176978429.90235801/v2

Lok, U. H., Lu, Z.-L., & Syu, J.-H. (2025). Forecasting option-implied dynamics with Google TimesFM: From model to market. *2025 IEEE International Conference on Big Data (BigData)*, 6914–6919.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). The M4 Competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34(4), 802–808. https://doi.org/10.1016/j.ijforecast.2018.06.001

McKenzie, E. D. (1984). General exponential smoothing and the equivalent ARMA process. *Journal of Forecasting,* 3(3), 333–344.

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers. *Eleventh International Conference on Learning Representations*.[C1]

Oldenburg, F., Han, Q., & Kaiser, M. (2024). Interpretable deep learning for forecasting online advertising costs: Insights from the competitive bidding landscape. In *2024 IEEE 11th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE.

Oreshkin, B. N., Carpov, D., Chapados, N., & Bengio, Y. (2020). N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. *Eighth International Conference on Learning Representations*.[C1]

Qi, Y., Hu, H., Lei, D., Zhang, J., Shi, Z., Huang, Y., Chen, Z., Lin, X., & Shen, Z.-J. M. (2025). TimeHF: Billion-scale time series models guided by human feedback. *arXiv*. https://doi.org/10.48550/arXiv.2501.15942

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Blog, 1*(8), 9.

Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting, 36*(3), 1181–1191.

Tan, Y. F., Ong, L. Y., Leow, M. C., & Goh, Y. X. (2021). Exploring time-series forecasting models for dynamic pricing in digital signage advertising. *Future Internet, 13*(10), Article 241. https://doi.org/10.3390/fi13100241

Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., & Sahoo, D. (2024). Unified training of universal time series forecasting transformers. *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 53140–53164.

Würfel, M., Han, Q., & Kaiser, M. (2021). Online advertising revenue forecasting: An interpretable deep learning approach. In *2021 IEEE International Conference on Big Data (Big Data)* (pp. 1980–1989). IEEE.

Zhang, G. P. (2003). Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing, 50*, 159–175. https://doi.org/10.1016/S0925-2312(01)00702-0

Appendix A: Overall Model Performance Charts

This appendix contains the complete set of evaluation charts comparing every model configuration, including the statistical baselines, the gradient-boosted trees, the zero-shot foundation models and all fine-tuned variants, across the three business verticals. For each of the eight evaluation metrics, a heatmap reports performance for every model within each vertical and a bar chart reports the average across verticals with models sorted from best to worst.

All charts in this appendix use a common red-yellow-green traffic-light colour scale in which green always indicates better relative performance and red always indicates worse, regardless of the direction of the underlying metric: lower is better for the error metrics (MAE, RMSE, MAPE, SMAPE, MdAPE and WAPE), higher is better for the coefficient of determination (R-squared), and closest to zero is better for bias. Heatmap cells and bar labels report the underlying metric value itself rather than the colour score, and grey indicates a missing value. Where a chart's values span several orders of magnitude the vertical axis is linear near zero and logarithmic beyond that point, as noted on the chart concerned.

Figure A1. Weighted Absolute Percentage Error (WAPE) by model and business vertical.

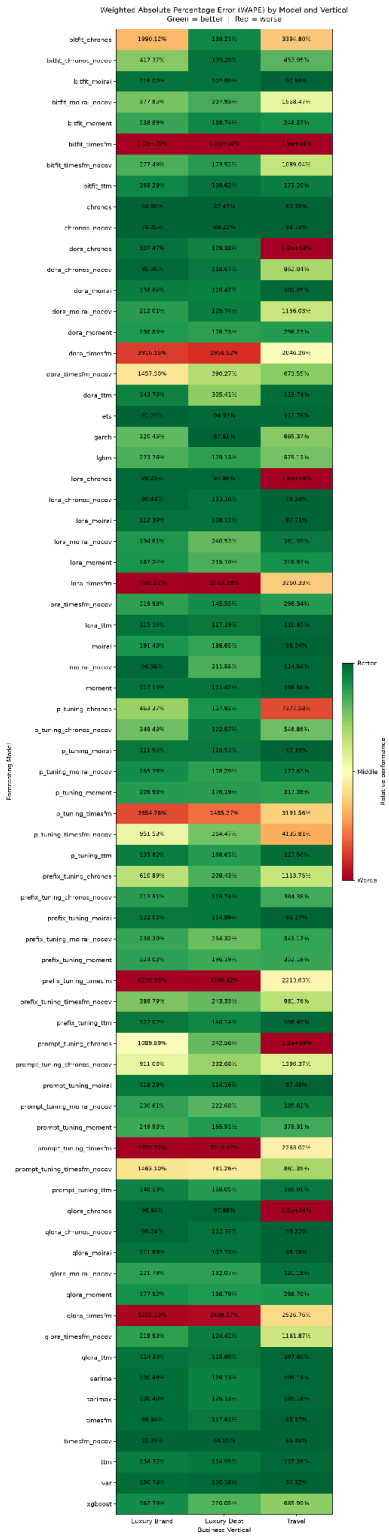


Figure A2. Weighted Absolute Percentage Error (WAPE) by model, averaged across verticals.

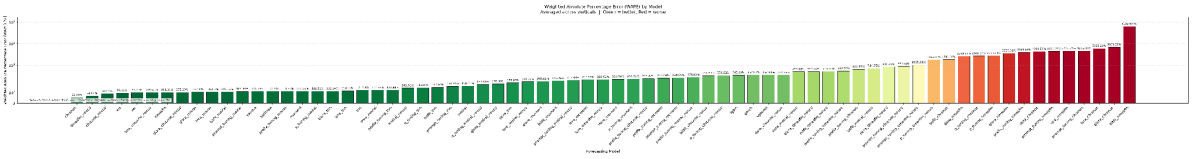


Figure A3. Median Absolute Percentage Error (MdAPE) by model and business vertical.

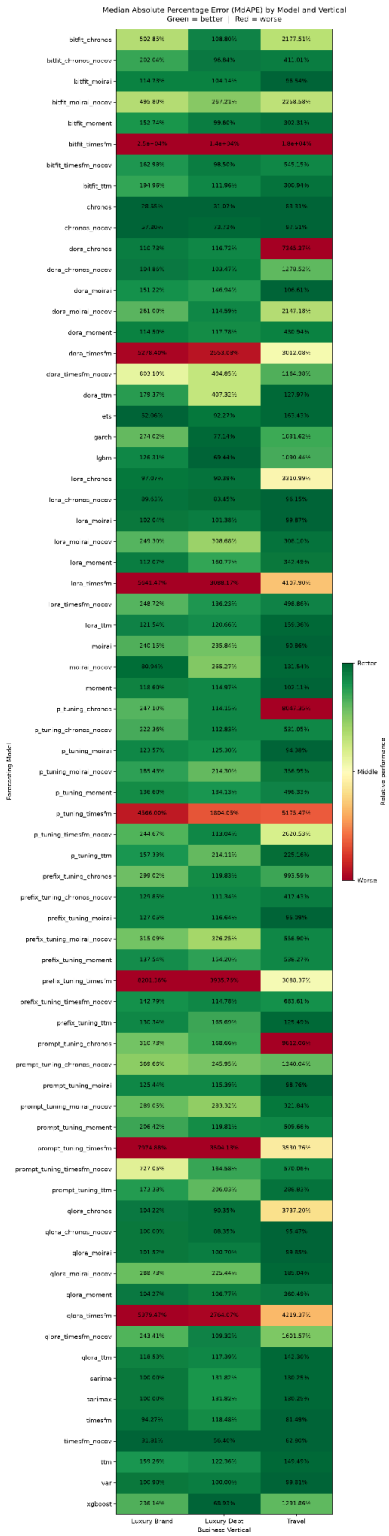


Figure A4. Median Absolute Percentage Error (MdAPE) by model, averaged across verticals.

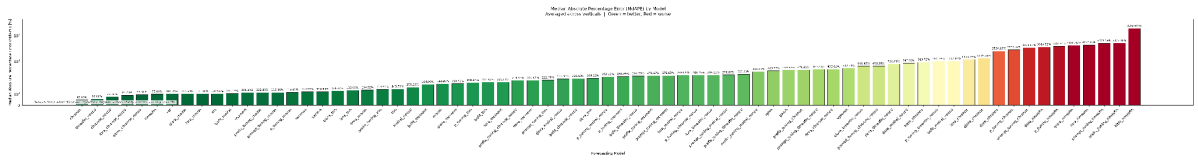


Figure A5. Mean Absolute Percentage Error (MAPE) by model and business vertical.

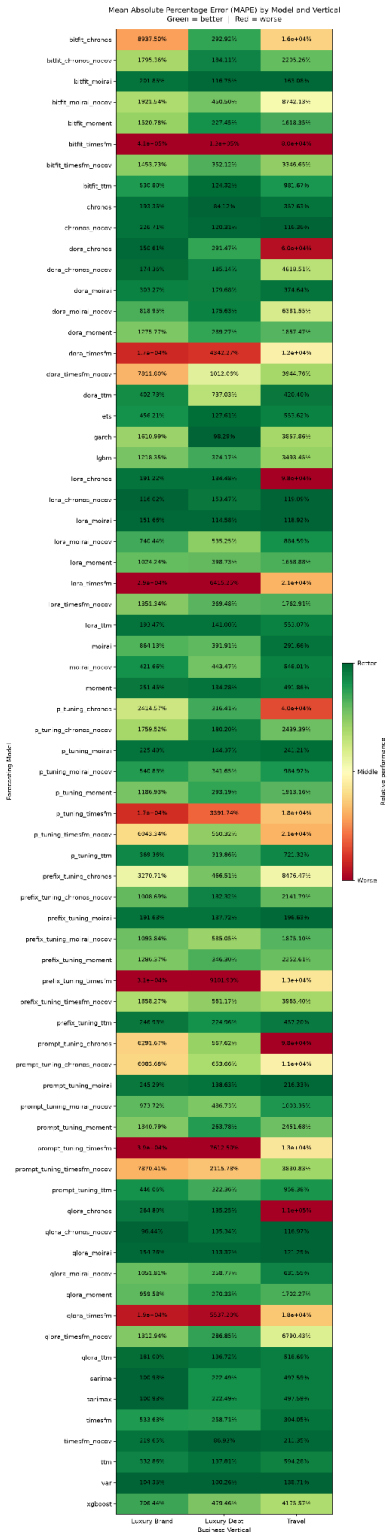


Figure A6. Mean Absolute Percentage Error (MAPE) by model, averaged across verticals.

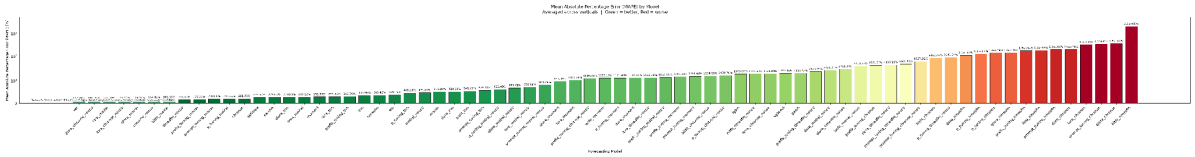


Figure A7. Symmetric Mean Absolute Percentage Error (SMAPE) by model and business vertical.

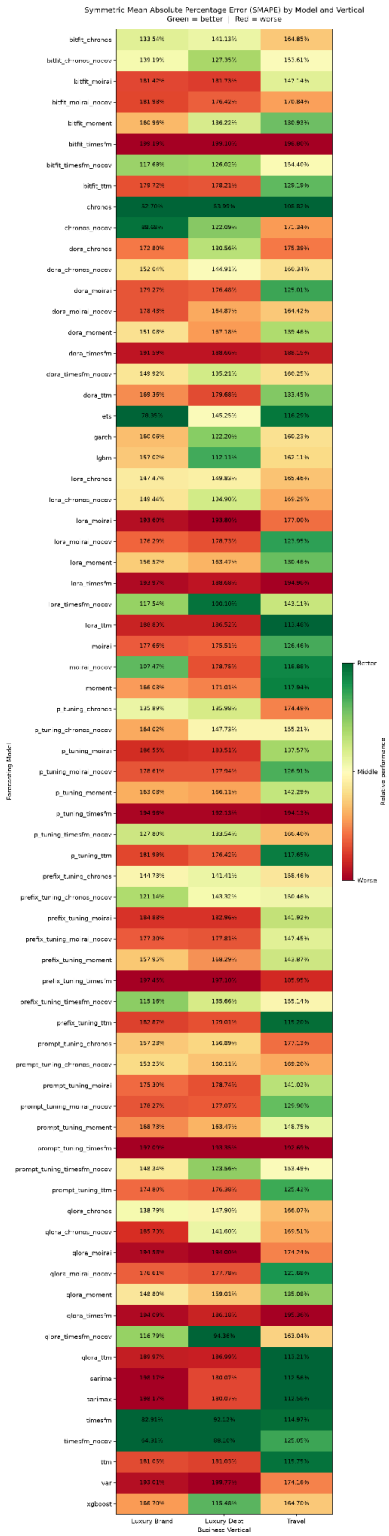


Figure A8. Symmetric Mean Absolute Percentage Error (SMAPE) by model, averaged across verticals.

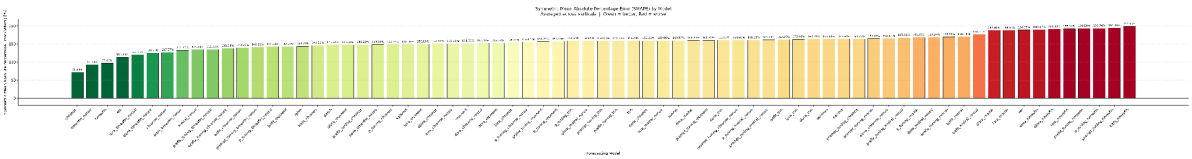


Figure A9. Mean Absolute Error (MAE) by model and business vertical.

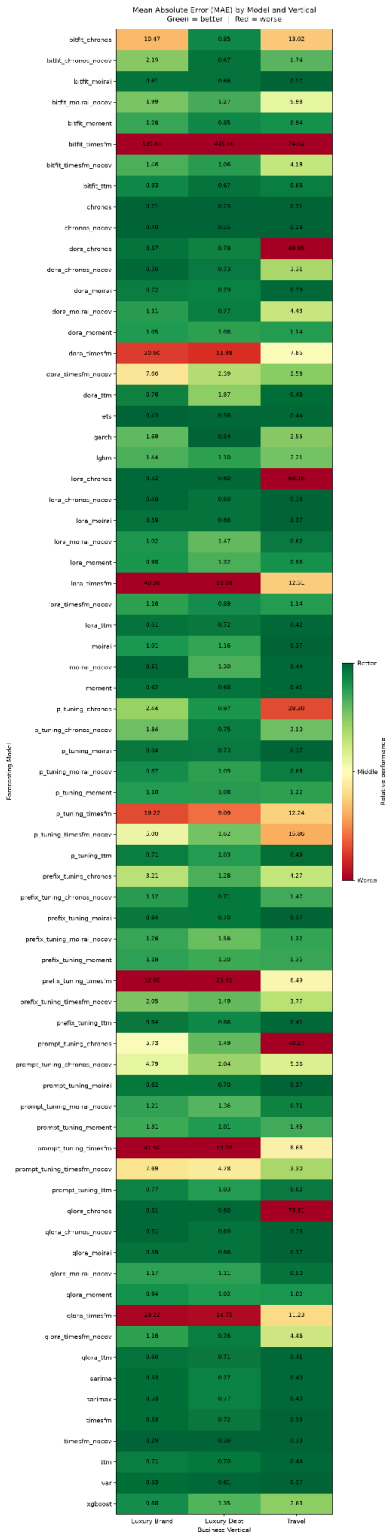


Figure A10. Mean Absolute Error (MAE) by model, averaged across verticals.

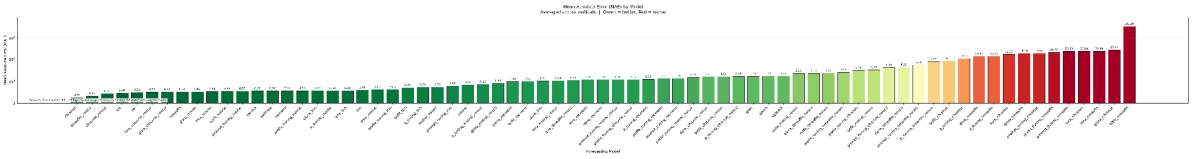


Figure A11. Root Mean Squared Error (RMSE) by model and business vertical.

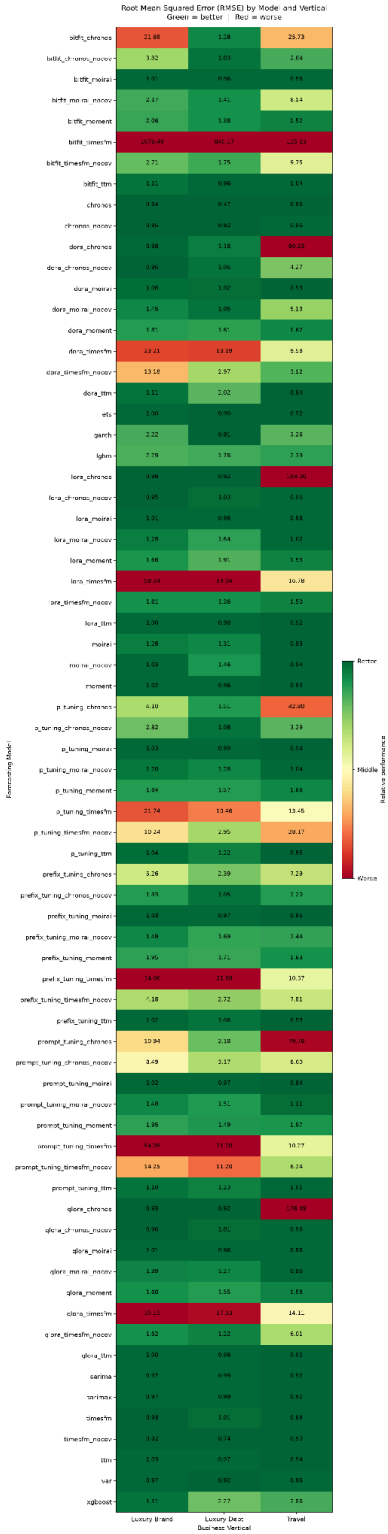


Figure A12. Root Mean Squared Error (RMSE) by model, averaged across verticals.

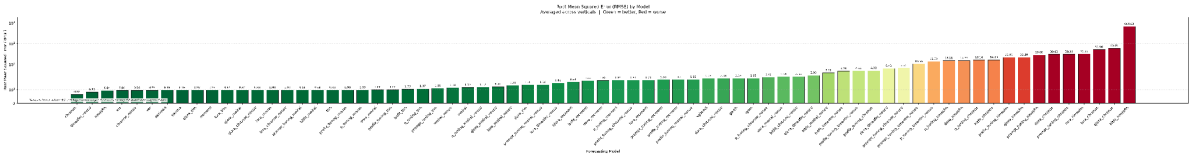


Figure A13. Coefficient of Determination (R-squared) by model and business vertical.

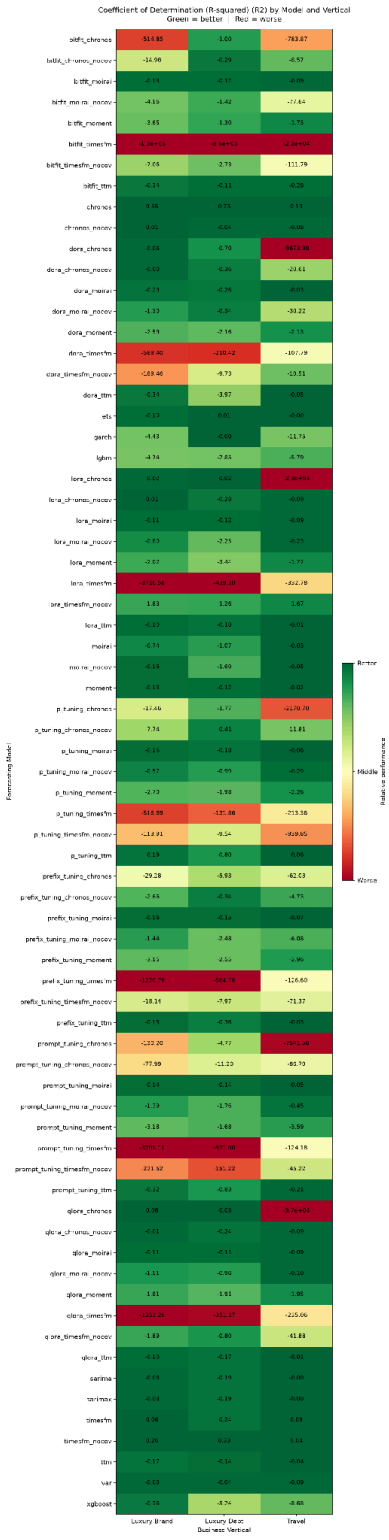


Figure A14. Coefficient of Determination (R-squared) by model, averaged across verticals.

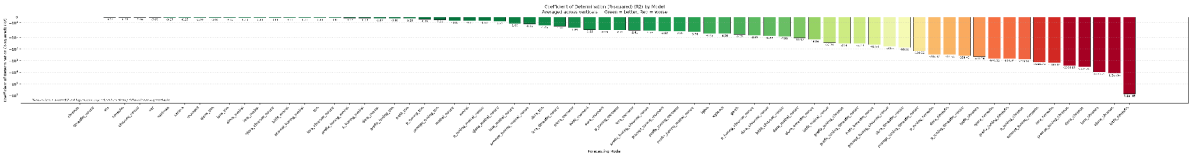


Figure A15. Mean Forecast Error (Bias) by model and business vertical.

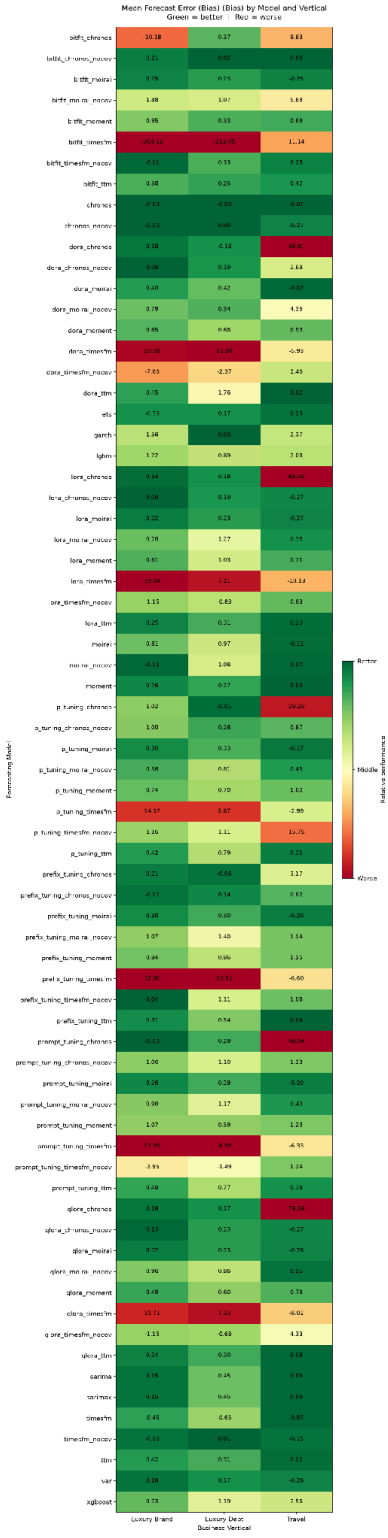


Figure A16. Mean Forecast Error (Bias) by model, averaged across verticals.

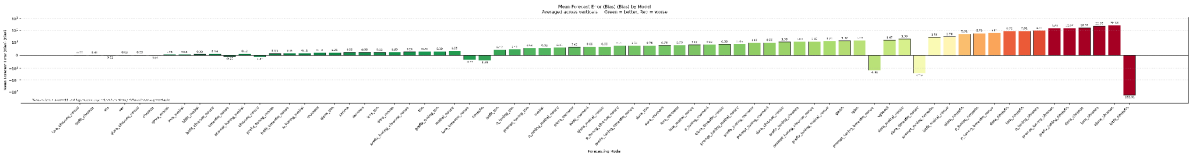
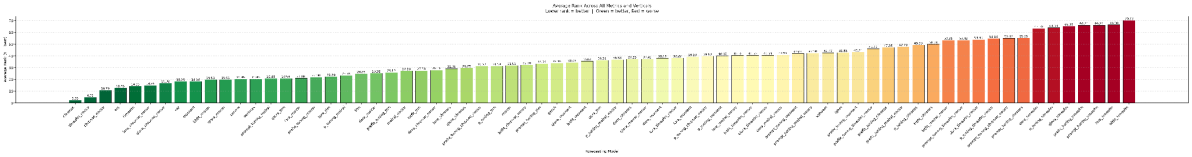


Figure A17. Average rank across all eight evaluation metrics and all verticals for every model configuration, where a rank of 1 is best. Reproduced from Figure 6.



Appendix B: Fine-Tuning Comparison Charts

This appendix contains the complete set of charts for the fine-tuning comparison, covering the foundation-model backbones and their variants under each of the seven parameter-efficient fine-tuning methods (LoRA, QLoRA, DoRA, BitFit, prompt tuning, prefix tuning and P-tuning). For each of the eight evaluation metrics there are three charts: a heatmap of per-vertical performance, a bar chart of the cross-vertical average, and an improvement chart expressing each fine-tuned variant's change relative to its own base model, in which positive values always indicate that fine-tuning improved that metric.

All charts in this appendix use a common red-yellow-green traffic-light colour scale in which green always indicates better relative performance and red always indicates worse, regardless of the direction of the underlying metric: lower is better for the error metrics (MAE, RMSE, MAPE, SMAPE, MdAPE and WAPE), higher is better for the coefficient of determination (R-squared), and closest to zero is better for bias. Heatmap cells and bar labels report the underlying metric value itself rather than the colour score, and grey indicates a missing value.

Where a chart's values span several orders of magnitude the vertical axis is linear near zero and logarithmic beyond that point, as noted on the chart concerned.

Figure B1. Weighted Absolute Percentage Error (WAPE) by model and business vertical.

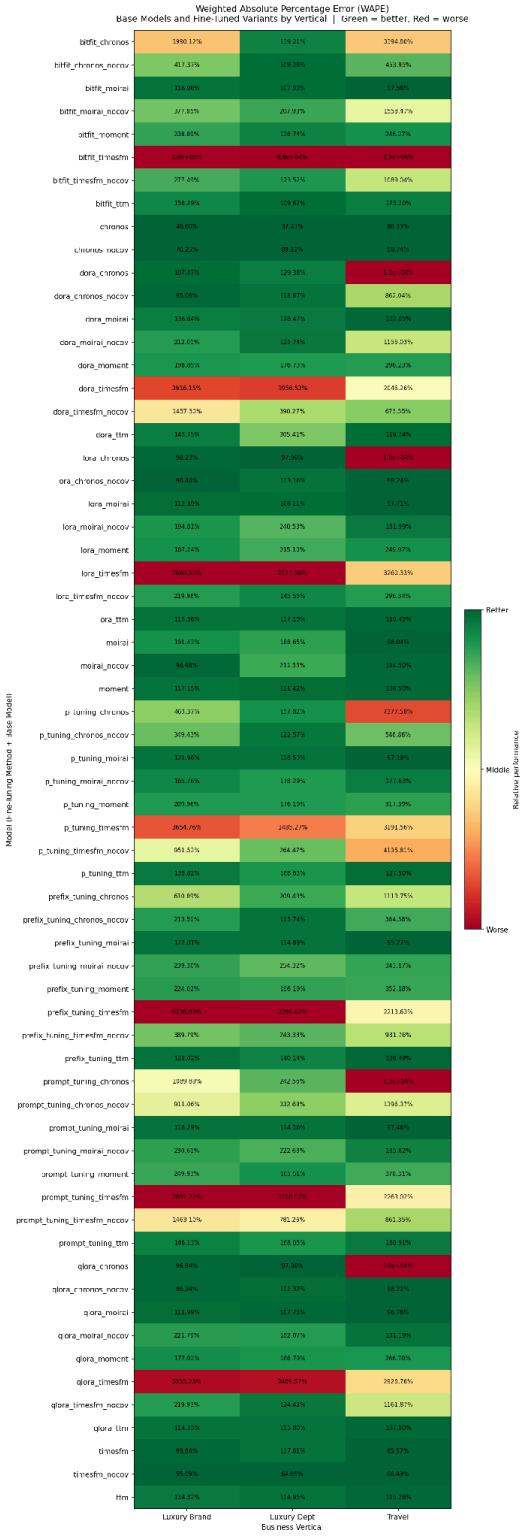


Figure B2. Weighted Absolute Percentage Error (WAPE) by model, averaged across verticals.

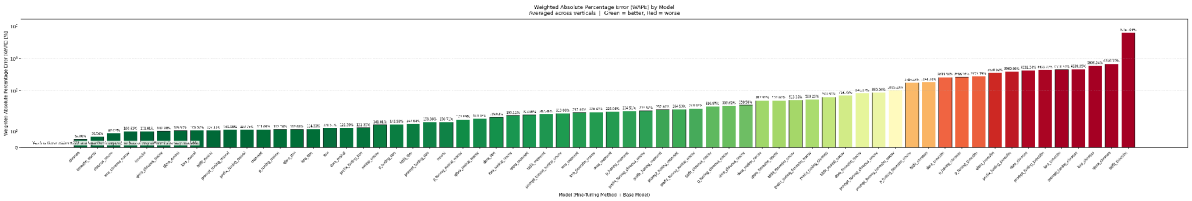


Figure B3. Improvement in Weighted Absolute Percentage Error (WAPE) from fine-tuning, for each fine-tuned model relative to its own base model.

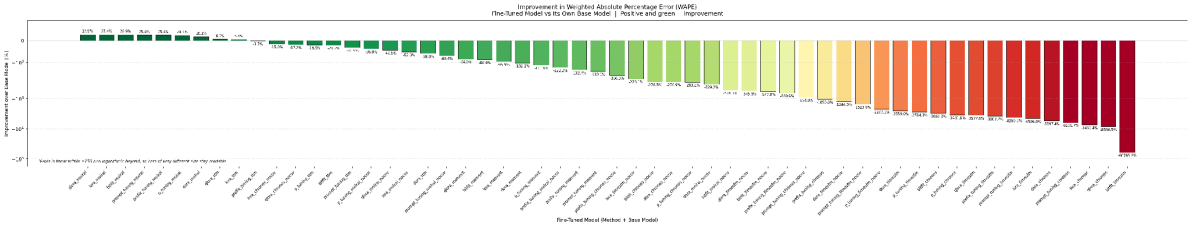


Figure B4. Median Absolute Percentage Error (MdAPE) by model and business vertical.

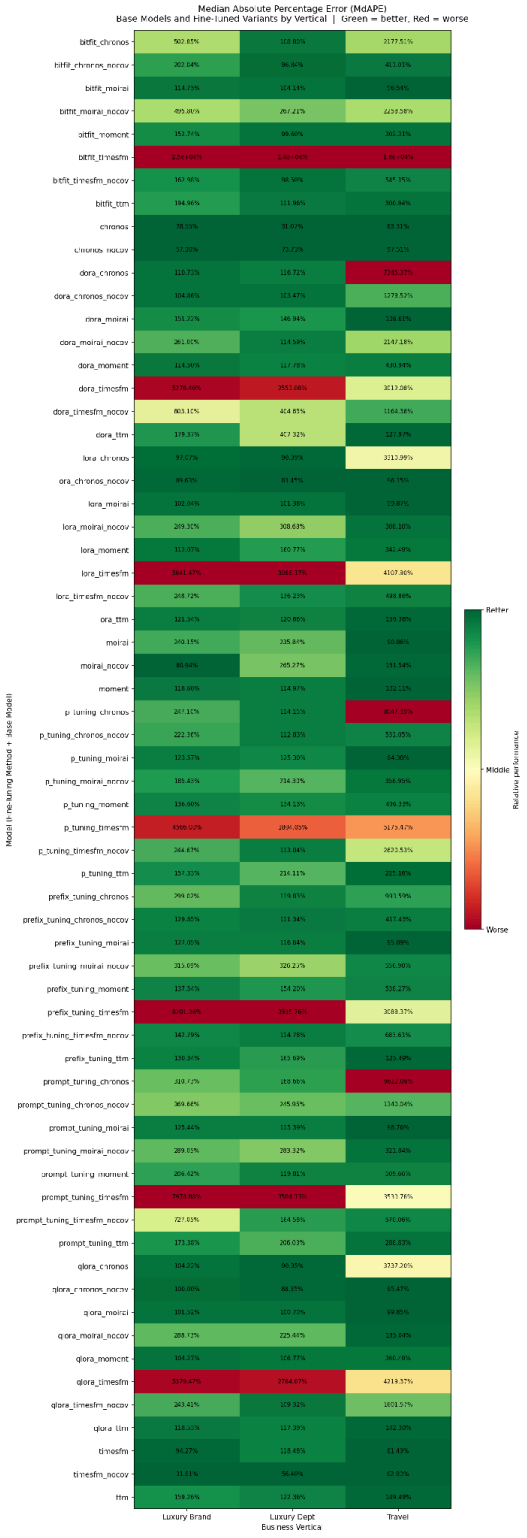


Figure B5. Median Absolute Percentage Error (MdAPE) by model, averaged across verticals.

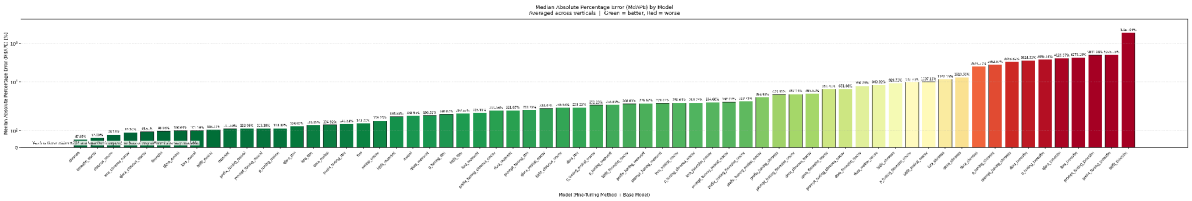


Figure B6. Improvement in Median Absolute Percentage Error (MdAPE) from fine-tuning, for each fine-tuned model relative to its own base model.

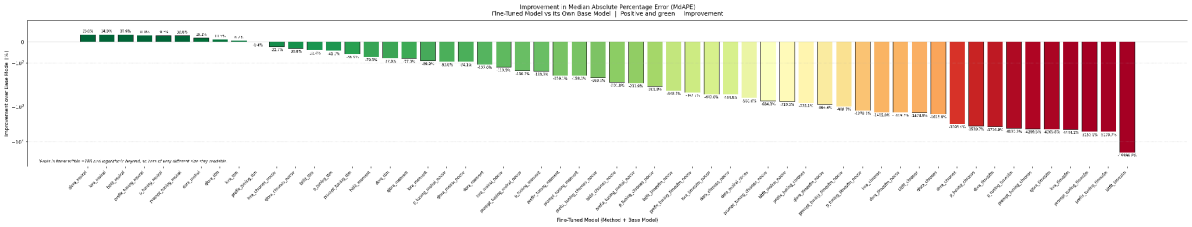


Figure B7. Mean Absolute Percentage Error (MAPE) by model and business vertical.

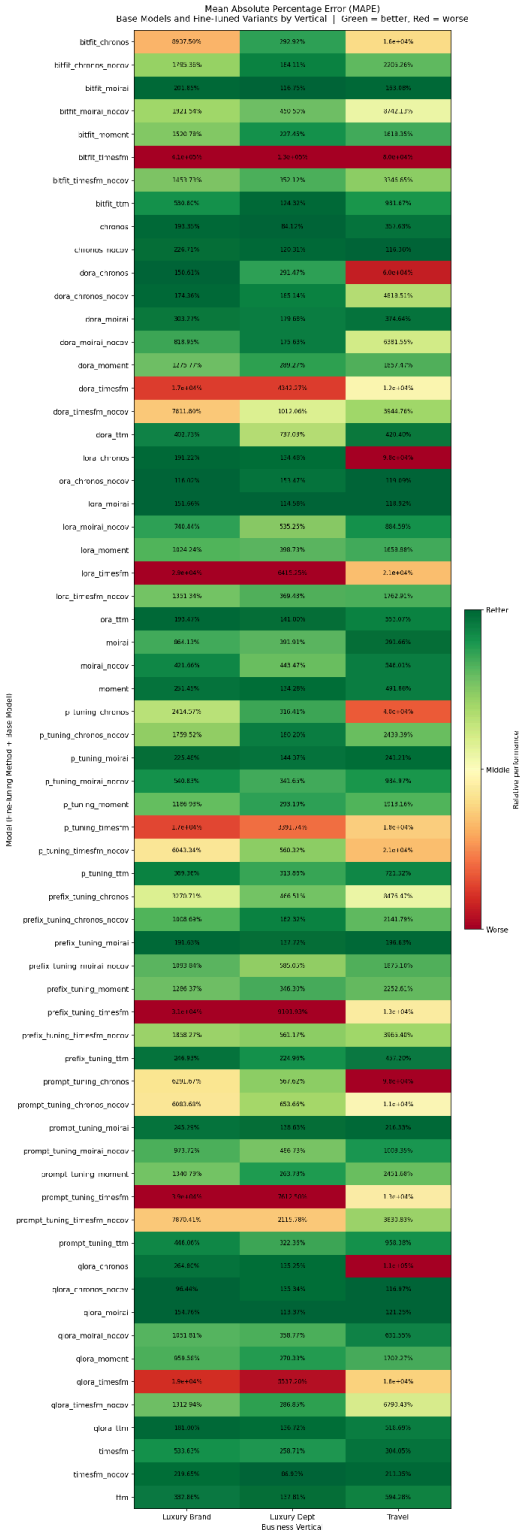


Figure B8. Mean Absolute Percentage Error (MAPE) by model, averaged across verticals.

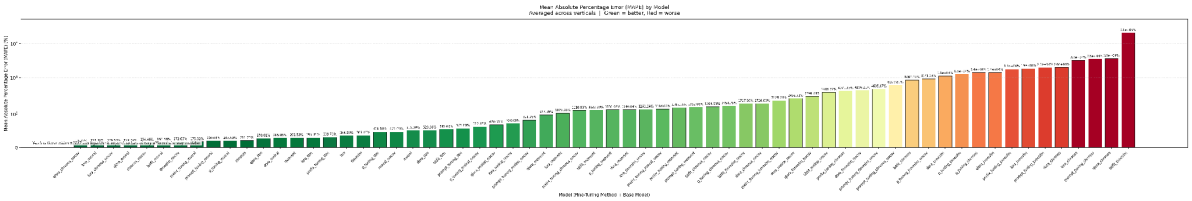


Figure B9. Improvement in Mean Absolute Percentage Error (MAPE) from fine-tuning, for each fine-tuned model relative to its own base model.

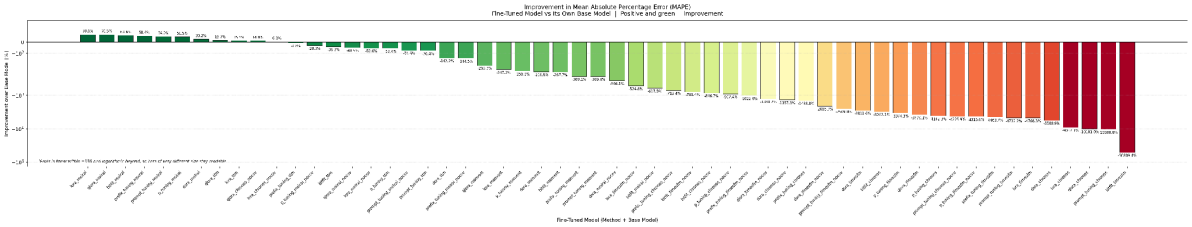


Figure B10. Symmetric Mean Absolute Percentage Error (SMAPE) by model and business vertical.

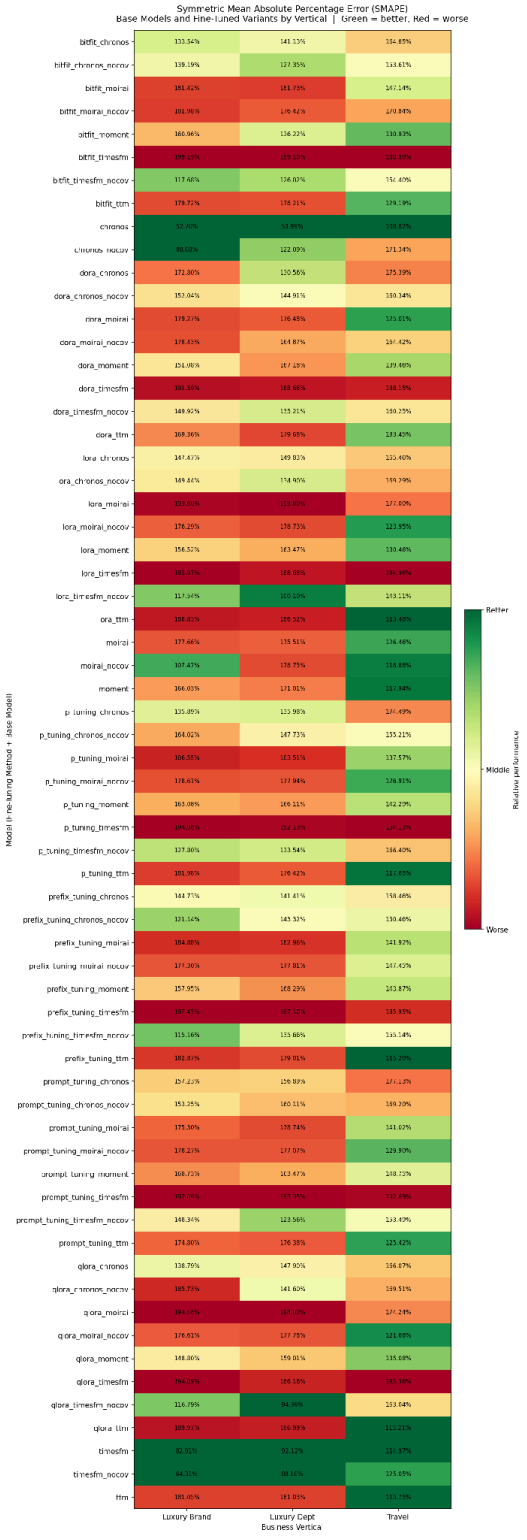


Figure B11. Symmetric Mean Absolute Percentage Error (SMAPE) by model, averaged across verticals.

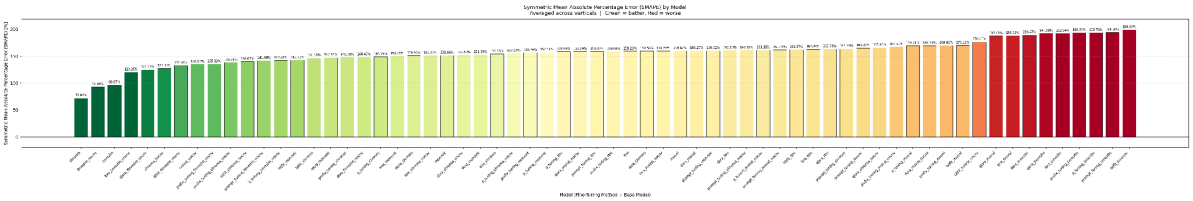


Figure B12. Improvement in Symmetric Mean Absolute Percentage Error (SMAPE) from fine-tuning, for each fine-tuned model relative to its own base model.

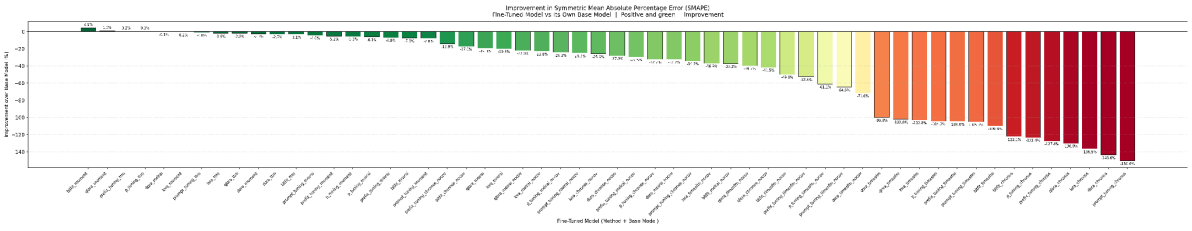


Figure B13. Mean Absolute Error (MAE) by model and business vertical.

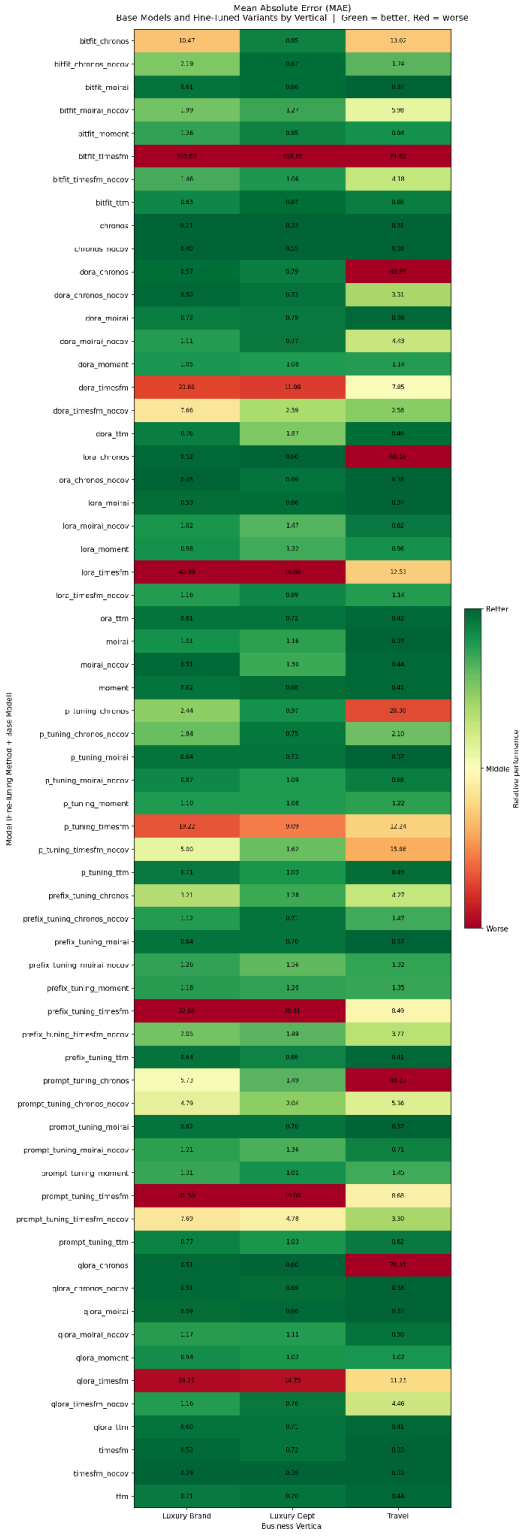


Figure B14. Mean Absolute Error (MAE) by model, averaged across verticals.

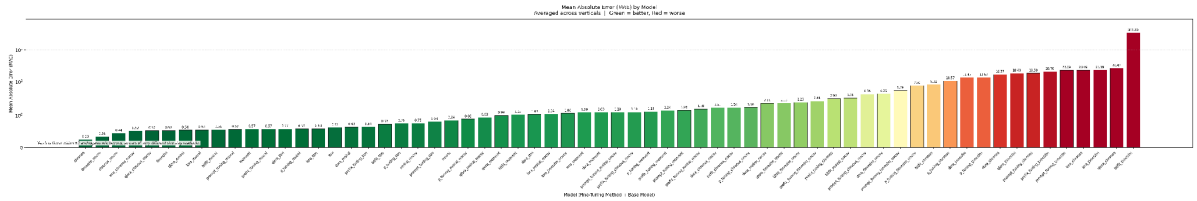


Figure B15. Improvement in Mean Absolute Error (MAE) from fine-tuning, for each fine-tuned model relative to its own base model.

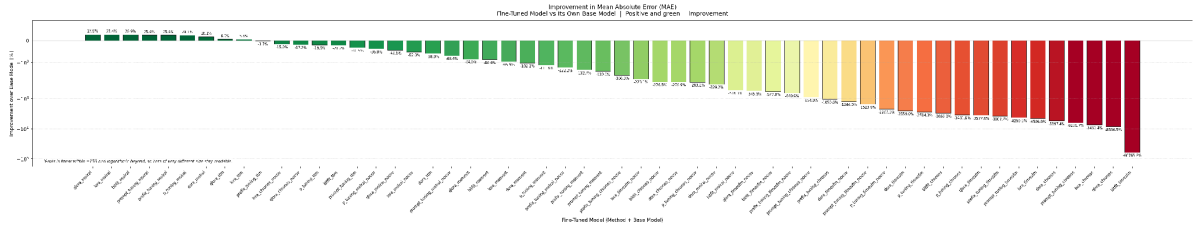


Figure B16. Root Mean Squared Error (RMSE) by model and business vertical.

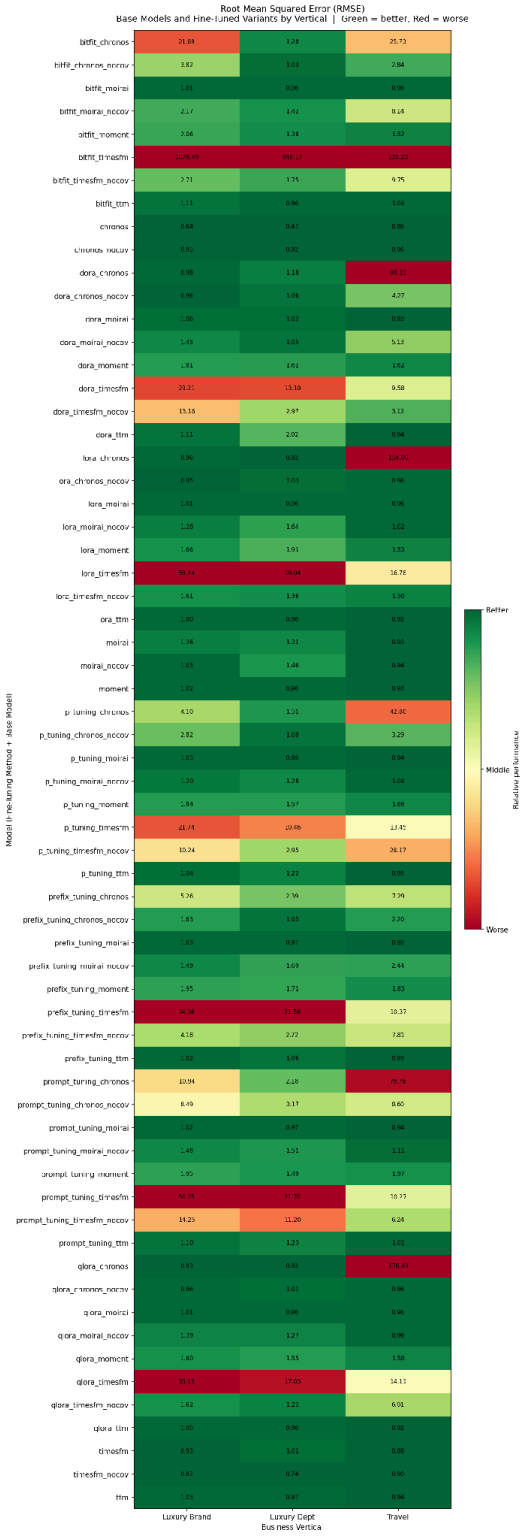


Figure B17. Root Mean Squared Error (RMSE) by model, averaged across verticals.

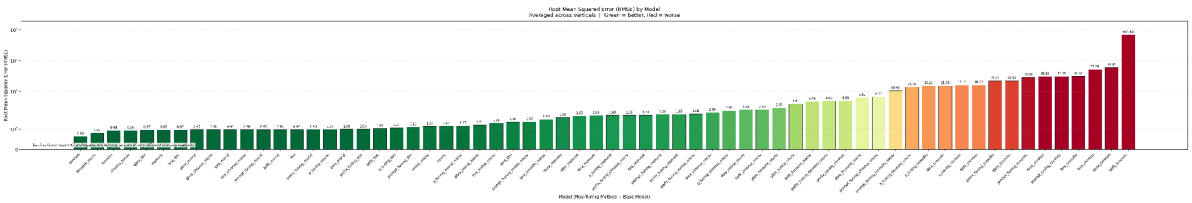


Figure B18. Improvement in Root Mean Squared Error (RMSE) from fine-tuning, for each fine-tuned model relative to its own base model.

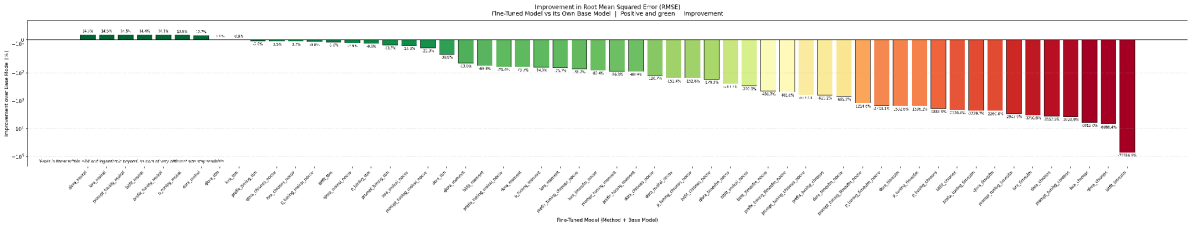


Figure B19. Coefficient of Determination (R-squared) by model and business vertical.

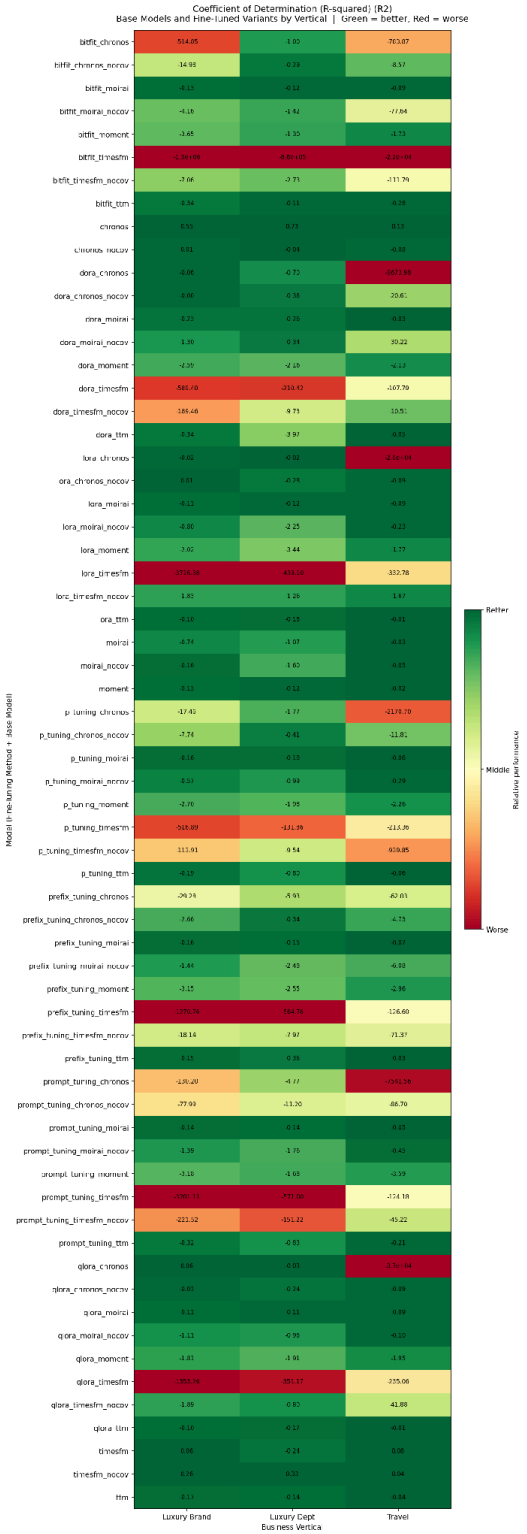


Figure B20. Coefficient of Determination (R-squared) by model, averaged across verticals.

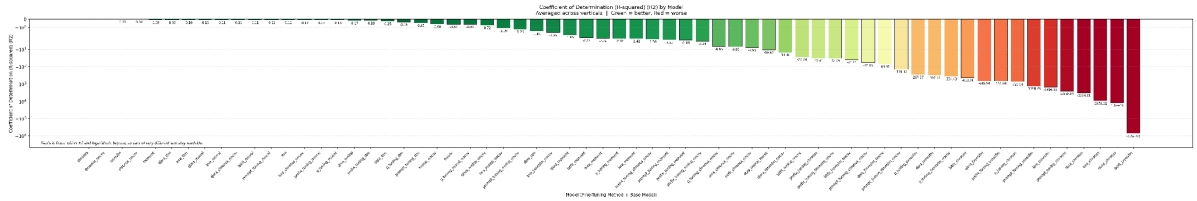


Figure B21. Improvement in Coefficient of Determination (R-squared) from fine-tuning, for each fine-tuned model relative to its own base model.

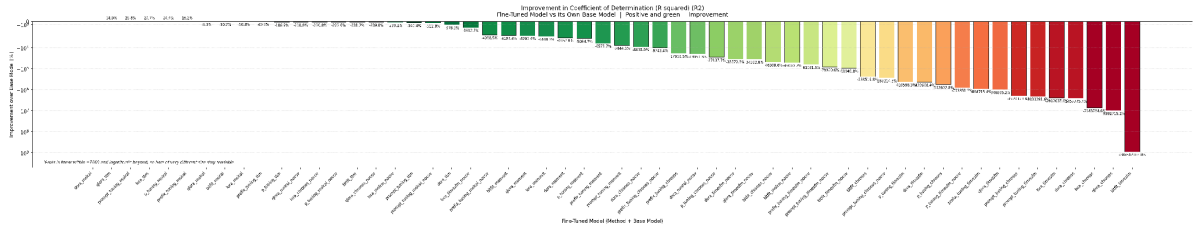


Figure B22. Mean Forecast Error (Bias) by model and business vertical.

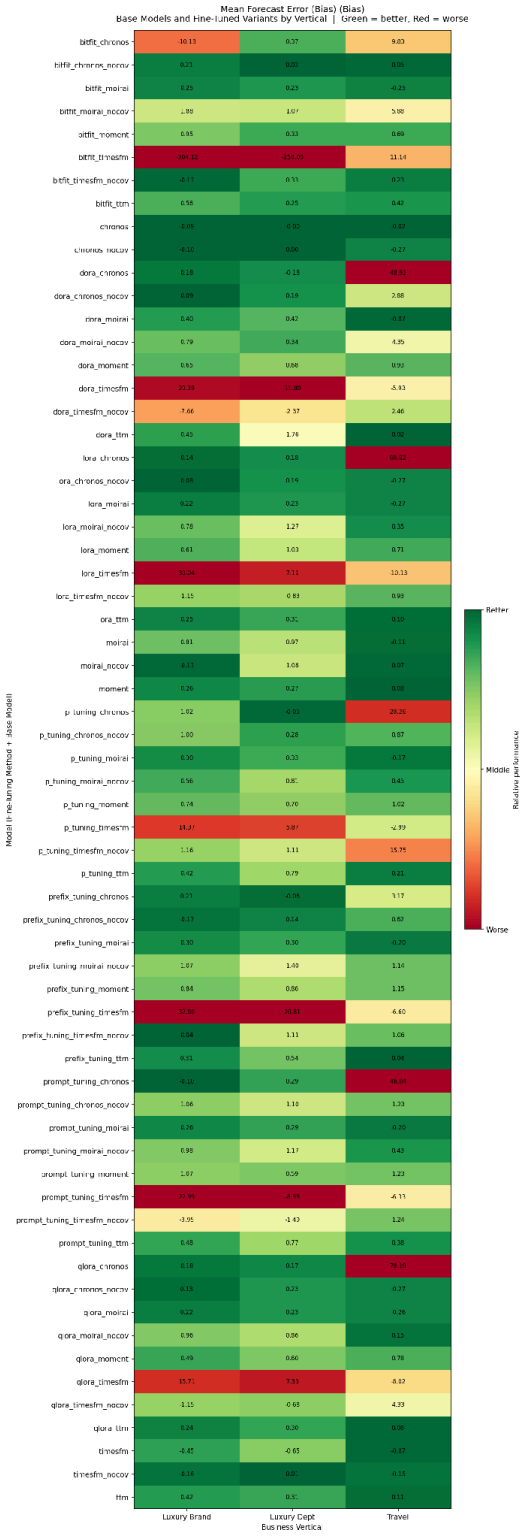


Figure B23. Mean Forecast Error (Bias) by model, averaged across verticals.

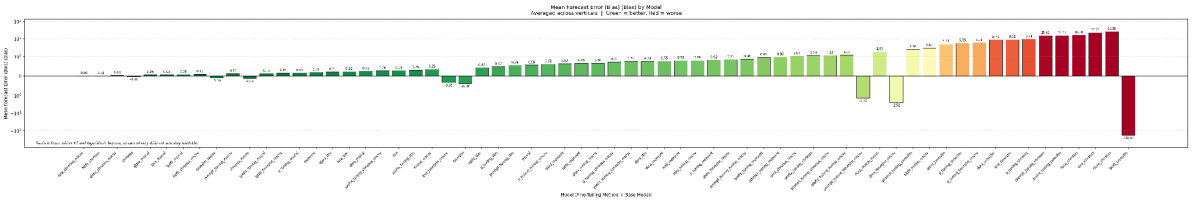


Figure B24. Improvement in Mean Forecast Error (Bias) from fine-tuning, for each fine-tuned model relative to its own base model.

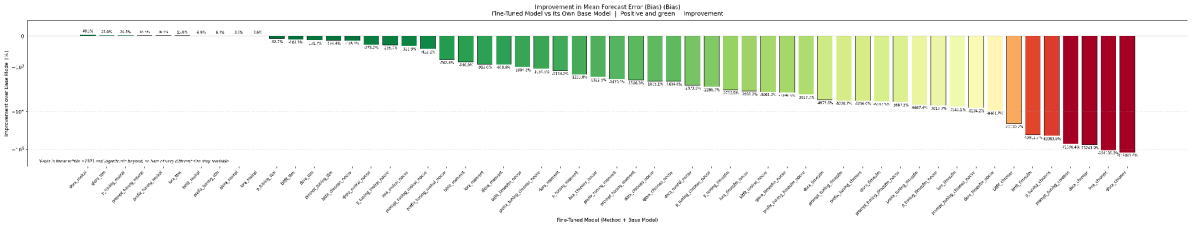
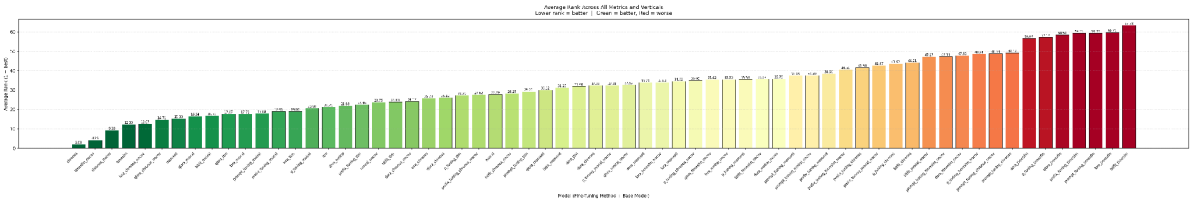


Figure B25. Average rank across all eight evaluation metrics and all verticals for the foundation models and their fine-tuned variants, where a rank of 1 is best.



Appendix C: Time Horizon Comparison Charts

This appendix contains the forecast-horizon comparison charts, which re-evaluate the models at four horizons (1 month, 3 months, 6 months and 12 months) using the same chronological train/test protocol described in the Methodology. The first three figures aggregate across the three business verticals; the final three break the same comparisons out by vertical.

In the line charts, each series is one model and lower values indicate better accuracy; the vertical axis is capped at 260% so that the competitive models remain distinguishable, and any series

running beyond that cap is identified in a note on the chart itself. In the fine-tuning success charts, the dashed reference line marks 50%, the level at which fine-tuning would help and harm equally often, so bars below that line indicate that fine-tuning degraded accuracy more often than it improved it.

Figure C1. Weighted Absolute Percentage Error (WAPE) by forecast horizon, averaged across the three business verticals. Reproduced from Figure 8.

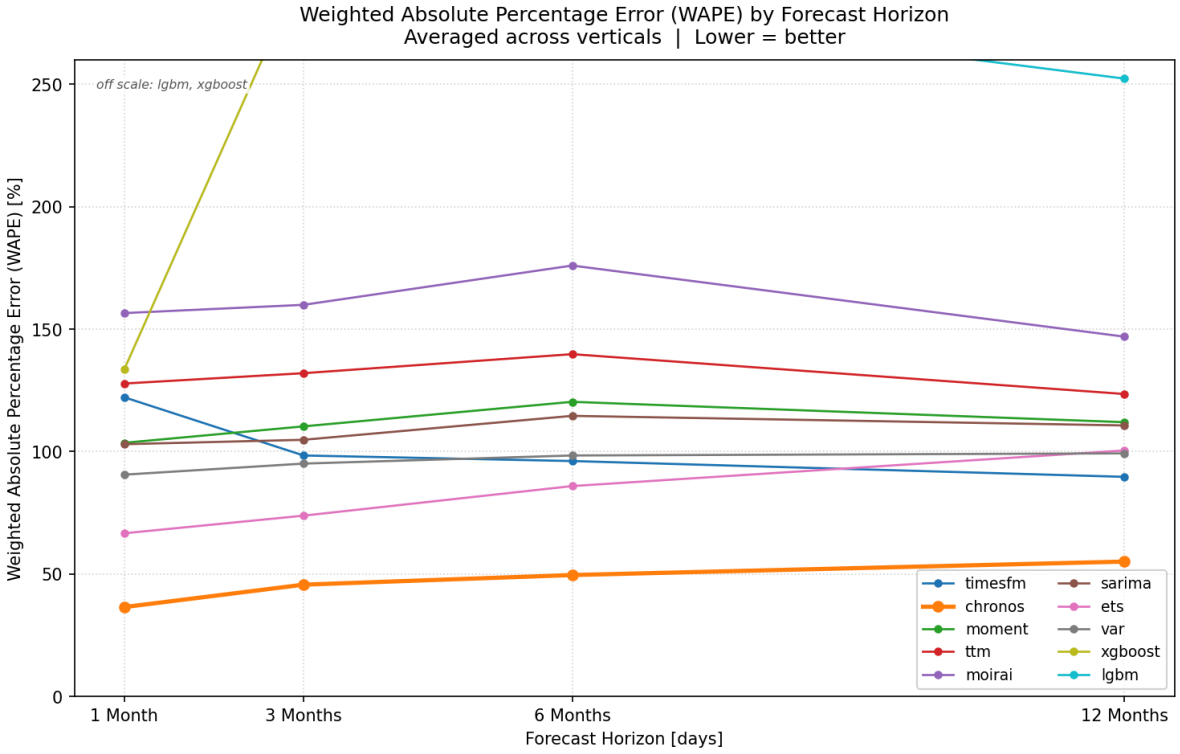


Figure C2. Median Absolute Percentage Error (MdAPE) by forecast horizon, averaged across the three business verticals.

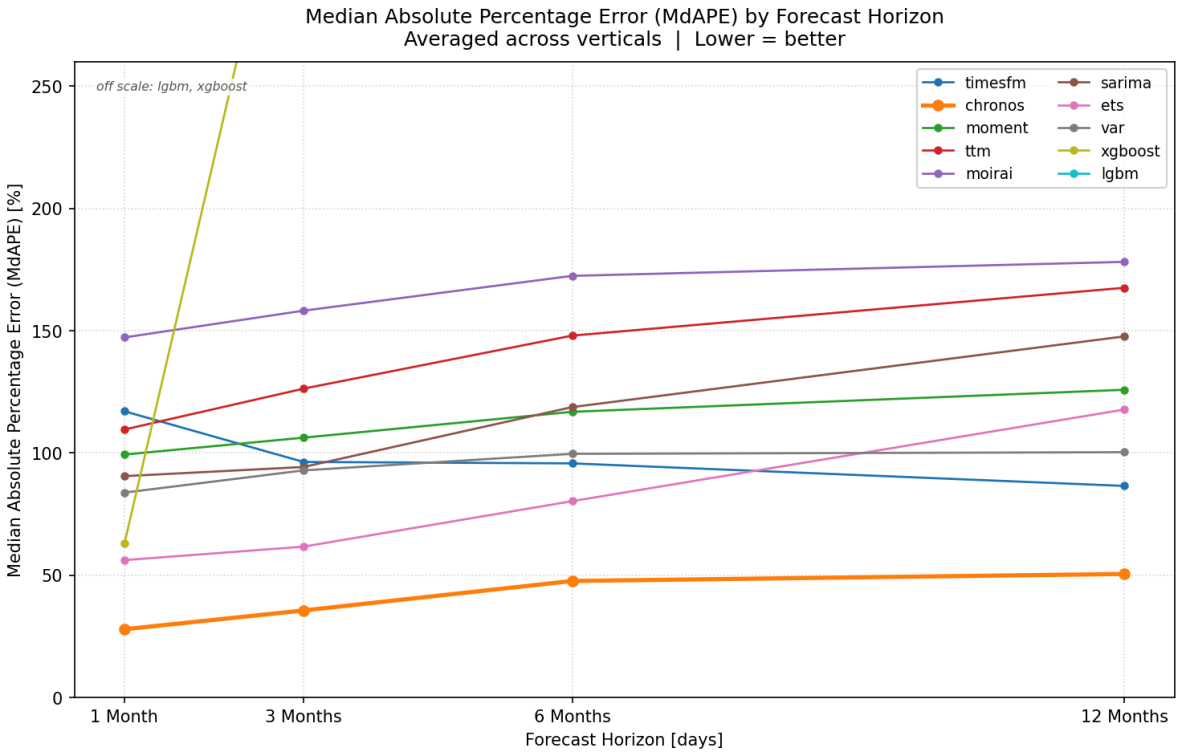


Figure C3. Share of fine-tuned runs that improved WAPE over their own base model, by forecast horizon. Reproduced from Figure 9.

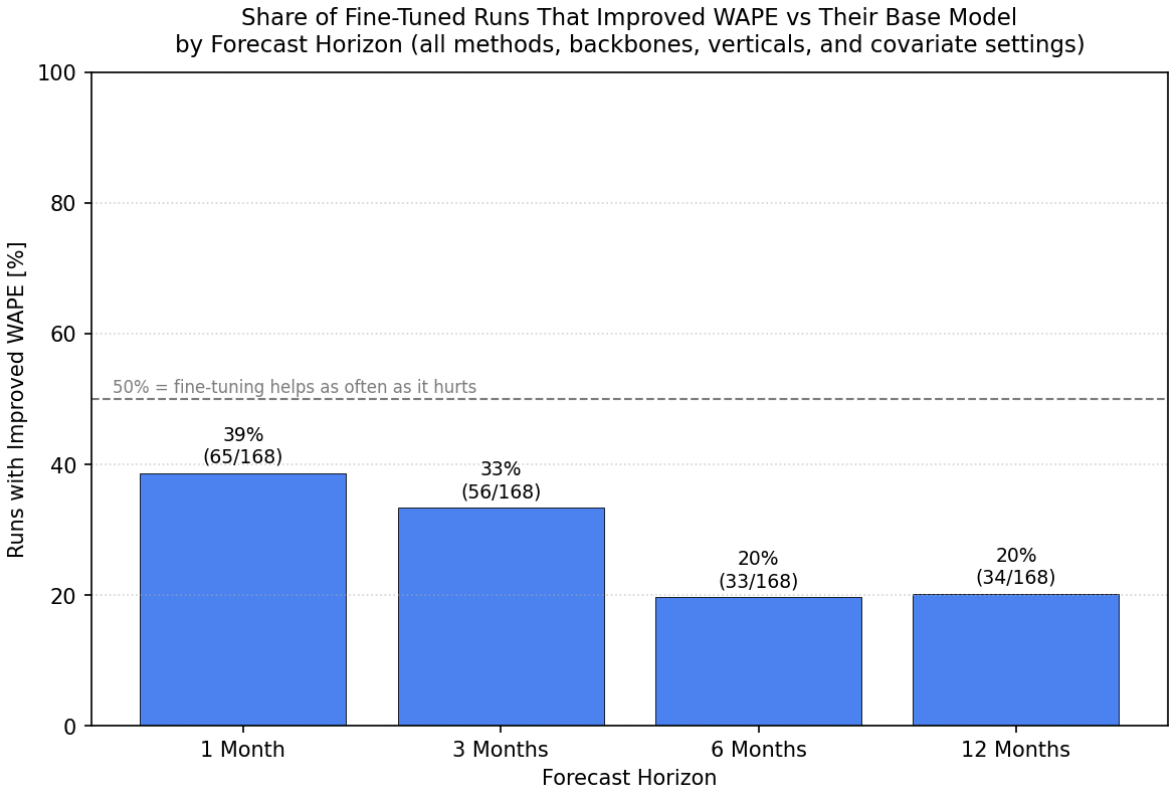


Figure C4. Weighted Absolute Percentage Error (WAPE) by forecast horizon, shown separately for each business vertical.

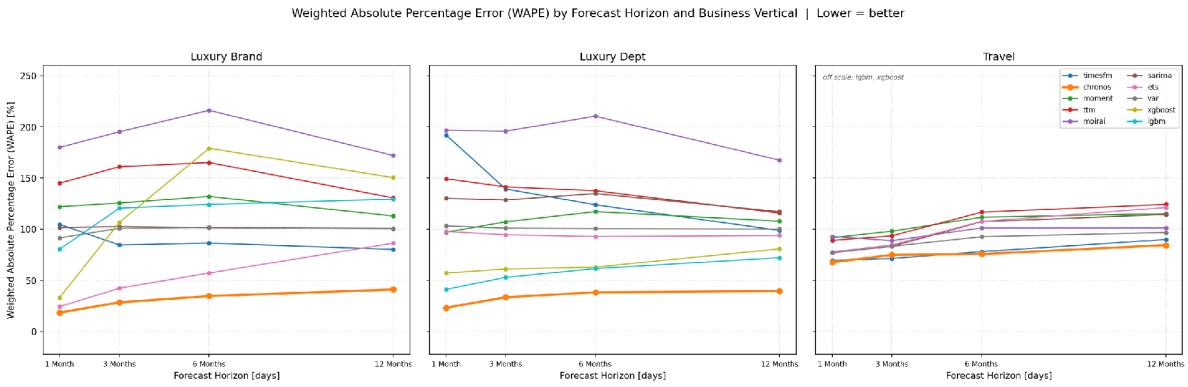


Figure C5. Median Absolute Percentage Error (MdAPE) by forecast horizon, shown separately for each business vertical.

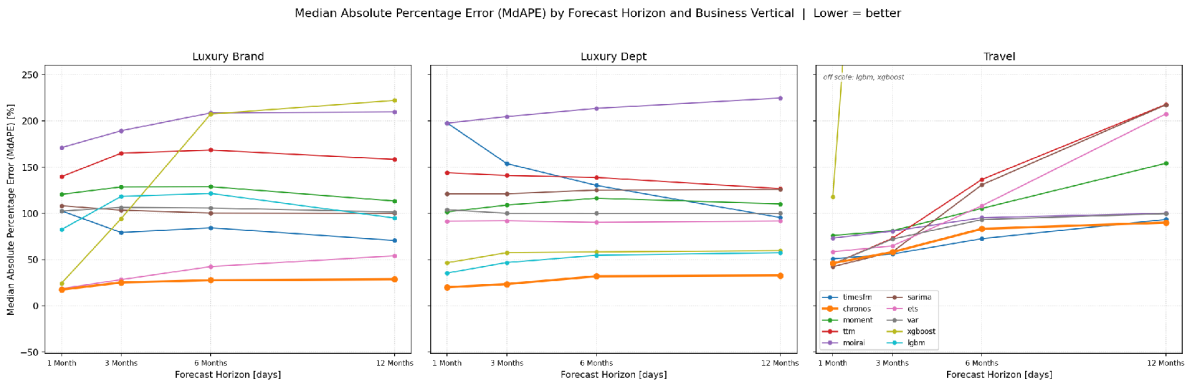
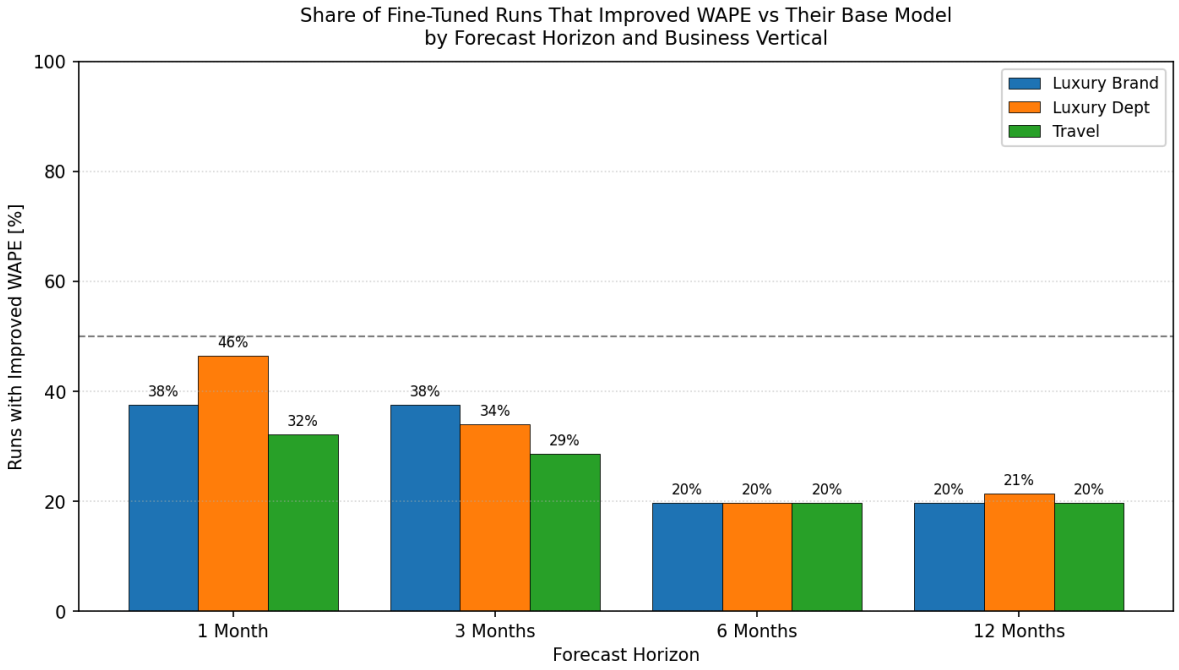


Figure C6. Share of fine-tuned runs that improved WAPE over their own base model, by forecast horizon and business vertical.



Appendix D: Cost Savings Calculation

Current cost:

400 forecasts/month × 12 = 4,800 forecasts/year

4,800 × 6 hours = 28,800 hours × $75 = $2.16M/year

Projected cost:

85% time reduction leaves ~1 hours per forecast for review

4,800 × 0.9 = 4,320 hours × $75 = $324K/year

Projected savings: $1.84M/year